# Highlights

**TRACE-MAS: Ranking What Agents See for Trustworthy Configuration-Evolved Multi-Agent Systems**

- We formulate agent-to-agent report sharing as a ranking problem for multi-agent systems.
- TRACE-MAS ranks reports, stores trace records, and self-revises routing decisions.
- A TRACE Score aligns assessment with grounding, robustness, hallucination risk, and cost.
- Experiments show fewer hallucination cascades and unsupported claims under noisy evidence.

# TRACE-MAS: Ranking What Agents See for Trustworthy Configuration-Evolved Multi-Agent Systems

**Abstract**

Large language model (LLM)-based multi-agent systems (MAS) show strong promise for complex reasoning, planning, and tool-augmented tasks. Recent automatic MAS design methods further reduce manual engineering by searching over agents, prompts, tools, model assignments, and communication topology. However, these methods mainly optimize how agents are organized, while leaving an important execution-time question under-specified: *which intermediate reports should downstream agents read?* As a result, unsupported reports can propagate as apparent evidence, causing false consensus and hallucinated final answers. We propose a *Trustworthy Routing and Assessment for Configuration-Evolved Multi-Agent Systems* framework (**TRACE-MAS**) that treats internal report selection as an explicit design problem. **TRACE-MAS** ranks candidate reports before downstream agents read them, stores trace records, scores configurations with a *TRACE Score*, and applies bounded self-revision to routing, verifier placement, context budgets, and scoring weights. Across *BBEH*, *WorkBench*, *SWE-Bench-Lite*, and *SWE-Bench-Verified*, **TRACE-MAS** improves EvoMAS LLM-selection baselines by 0.6–2.9% accuracy while remaining compatible with standard MAS evaluation. On *Noisy-BBEH-Mini*, full **TRACE-MAS** improves accuracy from 41.57% to 47.83% and reduces unsupported claims from 36.8% to 8.5%; under fluent unsupported reports, it reduces hallucination cascades from 64.8% to 18.5%. These results indicate that trustworthy MAS design must govern what agents see, not only how agents are connected or how strong their backbones are. Code is available in https://github.com/albert-jin/TRACE-MAS.

*Keywords:* Multi-agent systems, Large language models, Learning to rank, Hallucination mitigation, Retrieval-augmented generation, Trustworthy information access

## 1. Introduction

Large language models (LLMs) are increasingly used as agents that reason, call tools, observe feedback, and interact with external environments. *ReAct* shows how reasoning traces and actions can be interleaved for tool use [1], while *Voyager* studies skill accumulation through environment feedback [2]. *AgentBench* further frames agent evaluation as interaction with diverse environments rather than one-shot text generation [3]. This agentic view is especially important for information access. A modern search, recommendation, or decision-support workflow may involve query rewriting, retrieval, evidence checking, tool execution, and answer synthesis rather than a single retriever-generator call. As these workflows become longer and more interactive, the system must decide not only what external documents to retrieve, but also how intermediate evidence is organized for later reasoning.

Multi-agent systems (MAS) offer a natural way to organize such workflows. Role-based frameworks such as *CAMEL* and *AgentVerse* show how specialized agents can collaborate through structured dialogue [4, 5]. *AutoGen* provides a programmable conversation substrate for multi-agent applications [6], and software-oriented systems such as *ChatDev* and *MetaGPT* instantiate role-specialized development pipelines [7, 8]. Debate and peer-review workflows also show that multiple agents can improve reasoning when their interaction is carefully organized [9, 10].

Recent work further shows that MAS design itself can be automated. *AutoAgents* and *MAS-GPT* generate agent teams or executable MAS programs from task specifications [11, 12]. *ADAS* and *AFlow* search over agentic workflows represented as programs or graph-like structures [13, 14], while *EvoMAS* represents

MAS as structured configurations over agents, prompts, tools, communication topology, execution settings, and model assignments [15]. These advances make MAS more scalable and adaptable across tasks.

However, once a MAS starts running, another information-access problem appears inside the system. Each upstream agent produces intermediate reports, such as retrieved snippets, partial analyses, verifier comments, tool observations, or proposed answers. In retrieval-augmented generation, evidence quality and retrieval correction are known to affect final generation [16, 17]. In a MAS, the analogous evidence source is often produced by other agents rather than retrieved from a static corpus. Existing MAS design usually specifies who can communicate with whom, but it often leaves the content-level report-sharing rule implicit, even when topology or workflow search is explicit [6, 15]. If a communication path exists, broad previous context is simply passed forward. This is convenient, but it treats all visible reports as if they were equally suitable evidence.

*Such undifferentiated exposure is risky because agent-generated reports are not the same as static documents.* They can be fluent but unsupported, partially correct but misleading, or contradicted by other reports. Prior work on hallucination detection shows that fluent text can still lack evidential support [18]. Factuality and hallucination benchmarks make the same problem measurable at the claim and answer levels [19, 20]. When downstream agents receive all previous reports as ordinary context, a single unsupported report can be copied, paraphrased, reinforced by later agents, and finally accepted by the aggregator as apparent consensus. The failure is simple but important: *a risky intermediate report becomes visible context and is later reused as if it were evidence.*

This observation suggests that configuration-evolved MAS needs an explicit layer for controlling report visibility. Ranking has long governed which evidence a system exposes to users or models, from probabilistic retrieval to late-interaction neural retrieval [21, 22]. We bring this perspective inside MAS execution. The central question is simple: for a downstream agent, which subset of previous reports should be visible under a finite context budget? Passing all reports maximizes recall but increases risk. Passing too little context avoids noise but can remove useful collaboration. We therefore study report exposure as an optimization problem that balances task utility with support, consistency, robustness, risk, and cost.

We propose **TRACE-MAS**, a framework for trustworthy routing and assessment in configuration-evolved multi-agent systems. The proposed method treats internal report sharing as a ranking and assessment problem rather than as a passive consequence of topology. First, *exposure-aware routing* scores candidate reports and selects the reports shown to each downstream agent under topology, relevance, budget, and verifier constraints. Second, *trace records* store which reports are exposed, filtered, reused, and assessed. Third, *TRACE Score*-based assessment evaluates candidate configurations using both task and integrity signals. Fourth, *self-revision* updates routing and verification policies when the execution trace reveals risky report sharing. The resulting framework is not an open-ended search add-on. It revises the parts of a configuration responsible for unsupported reports reaching downstream agents.

Figure 1 contrasts all-context sharing with ranked report exposure. Without explicit control, unsupported intermediate reports may propagate across agents, be repeatedly reused, and eventually form a false sense of consensus. This example motivates treating report visibility as a design choice rather than treating all previous agent outputs as ordinary context.

We evaluate **TRACE-MAS** in both standard and noisy-evidence settings. On the standard main table, **TRACE-MAS** with LLM selection improves over *EvoMAS* LLM selection on *BBEH* (59.3 vs. 58.7), *WorkBench* (50.8 vs. 48.9), *SWE-Bench-Lite* (55.8 vs. 52.9), and *SWE-Bench-Verified* (65.0 vs. 63.8). We then use noisy-report settings to directly stress report sharing. On *Noisy-BBEH-Mini*, exposure-aware routing plus verification improves accuracy from 41.57 to 47.53 while reducing unsupported claims from 36.8% to 9.2% and noisy exposure from 29.4% to 7.0%. Under fluent unsupported reports, **TRACE-MAS** reduces hallucination cascade rate from 64.8% to 18.5% and false consensus from 42.3% to 10.2%. We further find that *GPT-5.5* and *Claude Opus 4.6* do not eliminate this risk: **TRACE-MAS** still improves over *Vanilla EvoMAS* by 8.7 and 9.6 points, respectively.

Our contributions are summarized as follows:

- We identify agent-to-agent report sharing as a missing ranking problem in MAS.
- We introduce **TRACE-MAS**, a framework that routes intermediate reports, stores trace records, and

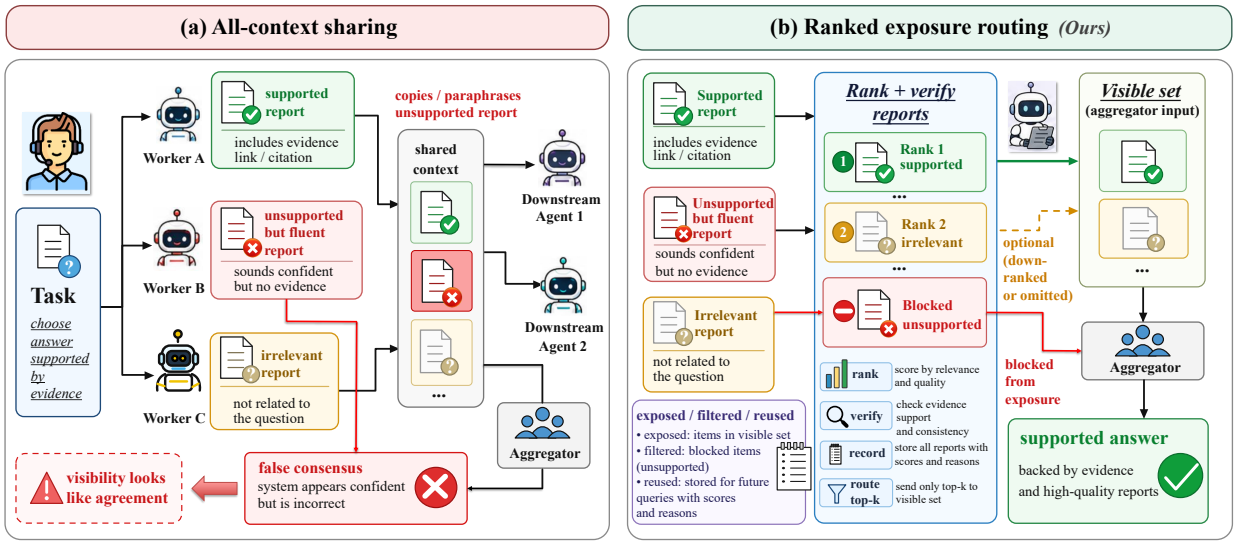



Figure 1: Motivating example. We contrast all-context sharing with ranked report exposure: unsupported intermediate reports may be propagated and treated as evidence by downstream agents, creating a false sense of consensus.

revises routing, verification, context budget, and scoring policies.

- We define a *TRACE Score* that aligns MAS assessment with evidence support, consistency, robustness, hallucination risk, unsupported claims, contradiction penalties, and cost.
- We design noisy-report, cascade, cost-integrity, transfer, trace-space, and strong-backbone evaluations for report sharing.
- We show that **TRACE-MAS** improves standard benchmark accuracy while reducing unsupported claims, noisy exposure, hallucination cascades, and false consensus. The code implementation is publicly available in https://github.com/albert-jin/TRACE-MAS.

**Roadmap.** The rest of the paper is organized as follows. Section 2 reviews LLM agents, automatic MAS design, and ranking-based evidence integrity. Section 3 presents a diagnostic study showing why report visibility matters before introducing the full method. Section 4 formulates the report-selection objective, defines *TRACE Score*, and describes routing, trace records, self-revision, and the risk-reduction intuition. Section 5 reports standard benchmark results, routing and assessment ablations, cost-integrity analysis, transfer, trace-space visualization, and strong-backbone robustness. Section 6 discusses method-level differences, limitations, and implications for trustworthy agentic information access.

## 2. Related Work

### *2.1. LLM Agents and Multi-Agent Collaboration*

LLM-based agents have been studied as interactive systems that combine language reasoning, tool use, environment feedback, and memory. ReAct shows how reasoning traces and actions can be interleaved for tool use [1]; Voyager demonstrates how an LLM agent can accumulate executable skills through environment feedback and self-verification [2]; and AgentBench evaluates agents across interactive environments [3]. These systems shifted LLM use from one-shot generation toward situated problem solving, where the model reads state, acts, observes consequences, and revises its behavior.

Multi-agent frameworks extend this idea by distributing work across roles. CAMEL and AgentVerse study role-based collaboration and group dynamics [4, 5], AutoGen provides a programmable multi-agent conversation framework [6], and ChatDev and MetaGPT instantiate software-development pipelines with

specialized agents and structured handoffs [7, 8]. Debate and peer-review workflows further show that multiple agents can improve reasoning when interaction is well structured [9, 10]. These systems demonstrate the value of agent specialization, but they also reveal a new reliability surface: an agent's output becomes another agent's input. Existing frameworks usually specify roles, message order, or communication topology, while treating the contents of visible intermediate reports as ordinary context. TRACE-MAS focuses on this missing layer. Even when the agent team is well designed, downstream agents still need a principled policy for which upstream reports deserve visibility.

### *2.2. Automatic MAS Design and Configuration Search*

Recent work studies how to automate agent workflow design. One line of work asks an LLM to generate or assemble agent teams. AutoAgents dynamically constructs agent roles and execution plans for a task [11], while MAS-GPT trains a model to generate executable multi-agent programs from query–MAS pairs [12]. These methods reduce manual engineering but can inherit the brittleness of generated natural-language or code-level system specifications.

Another line treats workflow construction as search. EvoAgent expands single-agent frameworks into multi-agent systems through evolutionary operators [23]; ADAS searches over agentic system designs represented as executable programs [13]; and AFlow formulates agentic workflow generation as search over code-represented workflows with LLM-invoking nodes and edges, using Monte Carlo Tree Search and execution feedback [14]. EvoMAS goes further in the MAS direction by representing systems as structured configurations over agents, prompts, tools, topology, execution settings, and model assignments [15]. This configuration view is important because it separates the logical MAS design from fragile code synthesis and makes model-role assignment an optimization dimension.

These methods make MAS construction more systematic, but their main decision variables are still pre-execution design choices: which agents to instantiate, which tools or prompts to use, which models to assign, and which communication paths to permit. They rarely treat the reports produced during execution as rankable evidence objects. A topology edge says that communication is possible, but it does not decide which reports deserve visibility, which repeated claims are unsupported, or when a downstream agent is seeing a misleading consensus. This leaves room for an internal ranking layer that decides which reports are shown after the MAS begins running.

### *2.3. Ranking, Retrieval, and Evidence Integrity*

Ranking has long been central to information retrieval, from probabilistic retrieval and BM25 to neural learning-to-rank and late-interaction retrieval models [21, 22, 24, 25]. Retrieval-augmented generation extends this idea to foundation-model generation by conditioning models on retrieved evidence [16]. Self-RAG and CRAG further show that retrieval should be adaptive and corrected when retrieved evidence is weak or misleading [17, 26]. TRACE-MAS extends this ranking perspective from external documents to internal agent reports. In an agentic workflow, intermediate reports function like evidence: they can support later reasoning, distract later agents, or introduce unsupported claims that are amplified by repetition.

This perspective also connects ranking to hallucination mitigation and evaluation. Work on SelfCheckGPT, FActScore, HaluEval, and RAGAS studies consistency, atomic factuality, hallucination labels, and RAG faithfulness metrics [18, 19, 20, 27]. LLM-as-judge evaluation, including G-Eval and MT-Bench, shows that strong LLM evaluators can be useful while still requiring calibration and bias awareness [28, 29]. TRACE-MAS does not assume that a final verifier alone is sufficient. We separate report routing, verifier insertion, and trace-based scoring because verification after a bad report has spread can be too late: a final judge may see coherent false consensus rather than the original evidence conflict. The benchmarks we use, including BBEH, WorkBench, and SWE-Bench, stress different versions of this problem across reasoning, workplace tool use, and software engineering [30, 31, 32].

## 3. Preliminary Analysis: Why Exposure Matters

Before introducing TRACE-MAS, we first ask a narrower diagnostic question: if the agents, topology, backbone, and task remain fixed, does the set of intermediate reports visible to downstream agents change

system behavior? This question is important because many MAS pipelines treat intermediate reports as a passive context pool. In that view, a report is useful if a later agent can read it, and the main design problem is how many agents to create or which backbone to assign. We test a different premise. In a multi-agent system, *who sees which report* is itself a ranking and integrity problem.

### *3.1. Motivation and Experimental Design*

In this preliminary study, we do not evaluate the full TRACE-MAS pipeline. Instead, we keep the base MAS fixed and change only the internal report-sharing policy. This isolates report visibility as a causal design variable before adding trace records, TRACE Score assessment, verifier checkpoints, or self-revision. The study therefore answers a mechanism question rather than a leaderboard question: can an untrusted intermediate report become harmful merely because the sharing policy makes it visible?

Thus, we compare four report-sharing policies. *All-context* shows every upstream report to downstream agents. *Direct-parent* shows only reports from topological parent agents. *Random top-$k$* limits context size by randomly selecting $k$ reports. *Ranked top-$k$* selects the top-$k$ reports according to evidence relevance. We run these policies on BBEH-Mini, WorkBench, and noisy variants where irrelevant, contradictory, fluent unsupported, or partially correct reports are injected into intermediate MAS traces. The main comparison uses the Vanilla EvoMAS selected configuration with Claude-3.5-Sonnet (C-3.5S). We measure task accuracy, unsupported-claim rate (UCR), noisy-report exposure (NER), evidence precision (EPrec), and token cost.

### *3.2. Report Sharing Changes Reliability*

Table 1 summarizes the key comparisons from this diagnostic study. On clean BBEH-Mini, all-context sharing obtains 49.07% accuracy, while ranked top-$k$ with $k = 2$ obtains 47.43%. This small clean-accuracy loss is not the main finding. The important change is that UCR drops from 19.4% to 6.9%, evidence precision rises from 71.6% to 83.7%, and token cost falls from 952K to 631K. Thus, even when the task is clean, unrestricted report sharing produces more unsupported final claims than a ranked report-sharing policy.

The noisy setting makes the mechanism sharper. On Noisy-BBEH-Mini with the same base configuration and backbone, all-context sharing drops to 41.57% accuracy, while UCR rises to 36.8% and NER reaches 29.4%. Direct-parent routing reduces the two integrity errors, but it relies on topology rather than report quality and can miss useful cross-branch evidence. Random top-$k$ shows that truncating context is not enough: it obtains only 40.43% accuracy and still leaves 22.4% UCR. Ranked top-$k$ is the only routing-only policy that improves all three headline diagnostic metrics at once: accuracy rises to 46.83%, UCR falls to 14.2%, and NER falls to 10.3%.

| Setting | Base MAS | Backbone | Sharing policy | Acc. | UCR | NER | EPrec | Tokens K |
|---|---|---|---|---|---|---|---|---|
| *Clean BBEH-Mini.* | | | | | | | | |
| Clean | EvoMAS config | C-3.5S | all-context | 49.07 | 19.4 | – | 71.6 | 952 |
| Clean | EvoMAS config | C-3.5S | direct-parent | 48.17 | 10.8 | – | 83.1 | 784 |
| Clean | EvoMAS config | C-3.5S | ranked top-$k$, $k = 2$ | 47.43 | **6.9** | – | **83.7** | 631 |
| *Noisy BBEH-Mini, 30% injected reports.* | | | | | | | | |
| Noisy | EvoMAS config | C-3.5S | all-context | 41.57 | 36.8 | 29.4 | 50.8 | 968 |
| Noisy | EvoMAS config | C-3.5S | direct-parent | 44.17 | 17.2 | 13.6 | 66.4 | 798 |
| Noisy | EvoMAS config | C-3.5S | random top-$k$, $k = 2$ | 40.42 | 22.4 | 17.2 | 57.6 | 632 |
| Noisy | EvoMAS config | C-3.5S | ranked top-$k$, $k = 2$ | 46.88 | 14.2 | 10.3 | 71.8 | 648 |
| Noisy | EvoMAS config | Q-235B | all-context | 47.23 | 37.5 | 30.4 | 49.8 | 1002 |
| Noisy | EvoMAS config | Q-235B | ranked top-$k$, $k = 2$ | **52.14** | 14.8 | 11.2 | 70.6 | 674 |
| *Noisy WorkBench.* | | | | | | | | |
| Noisy | EvoMAS config | C-3.5S | all-context | 38.17 | 34.5 | 28.6 | 51.8 | 7342 |
| Noisy | EvoMAS config | C-3.5S | ranked top-$k$, $k = 2$ | 43.43 | 14.6 | 10.8 | 71.5 | 4961 |
| Noisy | EvoMAS config | Mixed | all-context | 42.43 | 33.8 | 27.8 | 53.2 | 7627 |
| Noisy | EvoMAS config | Mixed | ranked top-$k$, $k = 2$ | **48.53** | **14.2** | **10.5** | **72.8** | 5178 |

Table 1: Preliminary report-sharing analysis. We keep the base MAS fixed and change only which intermediate reports are shown to downstream agents. The table reports mean accuracy, unsupported-claim rate (UCR), noisy-report exposure (NER), evidence precision (EPrec), and token cost.

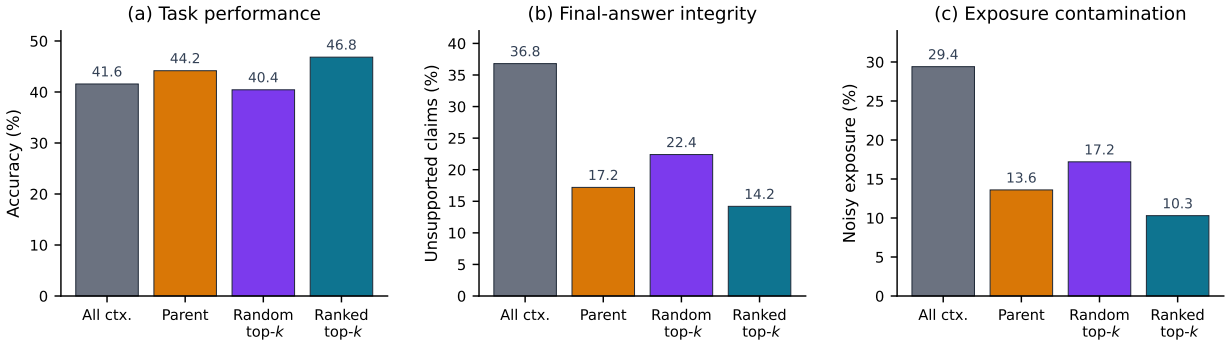



Figure 2: Preliminary report-sharing comparison on Noisy-BBEH-Mini with the Vanilla EvoMAS selected configuration and C-3.5S. Higher accuracy is better, while lower unsupported-claim rate and lower noisy-report exposure are better. Ranked top-$k$ is not the full TRACE-MAS system; it is a diagnostic policy showing that report visibility alone changes both task performance and integrity.

### *3.3. Findings and Transition to TRACE-MAS*

These results lead to three observations. First, all-context sharing is not a neutral implementation choice. It increases the chance that a misleading report becomes visible to several downstream agents and is later treated as consensus. Second, topology-only routing is safer than all-context sharing, but it cannot distinguish a weak parent report from a strong cross-branch report. Third, random truncation does not explain the gain of ranked exposure; the difference between random top-$k$ and ranked top-$k$ shows that evidence quality matters more than simply using fewer tokens.

The same pattern appears beyond the single BBEH setting. On Noisy-WorkBench with C-3.5S, ranked top-$k$ improves accuracy from 38.17% to 43.43%, reduces UCR from 34.5% to 14.6%, and reduces NER from 28.6% to 10.8%. Under mixed model selection, Noisy-WorkBench improves from 42.43% to 48.53%, while NER drops from 27.8% to 10.5%. These findings suggest that report-sharing policy and backbone strength are complementary: better models help, but they do not remove the need to control which reports become visible.

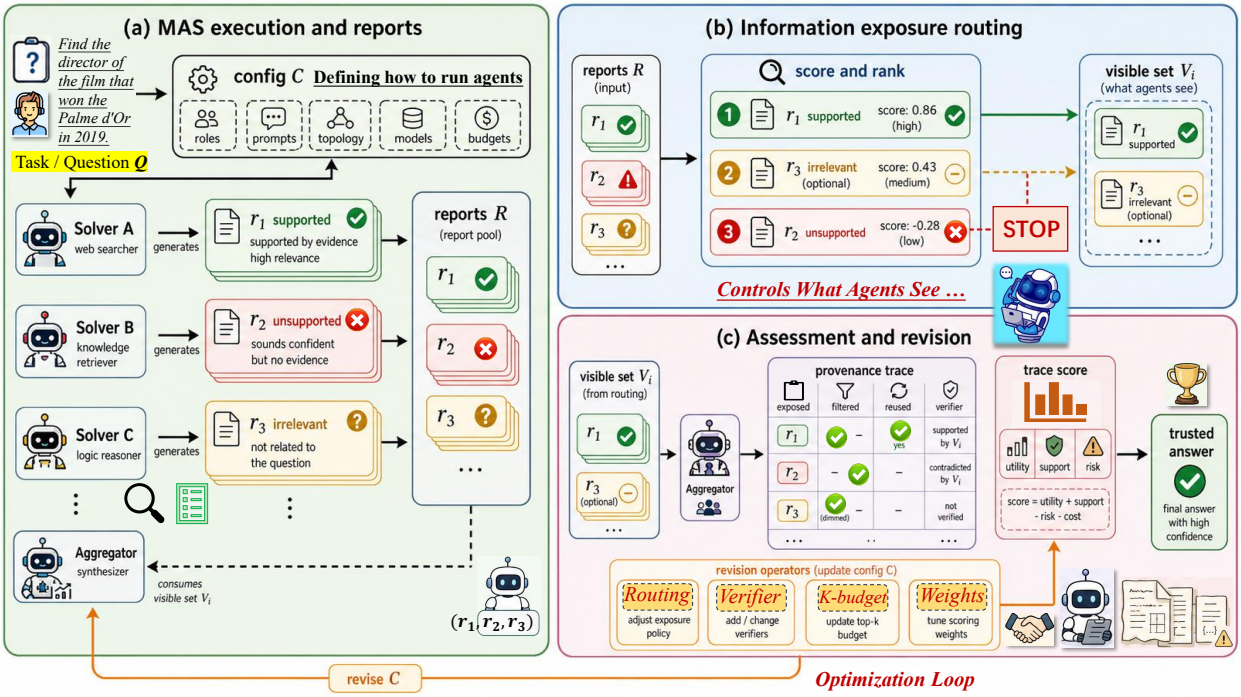



Figure 3: Our proposed TRACE-MAS framework. (a) MAS execution starts from a candidate configuration and produces intermediate reports, including supported, irrelevant, and unsupported evidence. (b) Exposure-aware routing scores candidate reports, selects the reports shown to downstream agents, and blocks risky reports before aggregation. (c) Trace recording, TRACE Score assessment, and self-revision close the loop by updating routing, verification, budget, and scoring policies from recorded failures.

**Preliminary hypothesis.** Reliable MAS reasoning requires more than better agents or larger context windows. We hypothesize that agent-to-agent report access should be controlled by ranking and support checks: each downstream agent should see a small task-relevant subset of reports, and the system should assess configurations not only by task accuracy but also by whether their traces suppress unsupported evidence propagation.

Motivated by this hypothesis, TRACE-MAS makes report sharing a controlled part of MAS design. The next section introduces four pieces: a representation for reports shown to downstream agents, trace records that store who saw what, a TRACE Score that balances task utility with integrity risk, and a self-revision loop that updates MAS configurations from observed failures.

## 4. TRACE-MAS

TRACE-MAS is a self-revision framework for controlling which intermediate reports are shown inside configuration-based MAS. As shown in Figure 3, our method starts from a MAS configuration, executes it on a task, records internal reports and routing decisions, scores the resulting trace with metrics that combine task success and evidence risk, and revises the configuration when the trace indicates unsafe report sharing. We organize TRACE-MAS into four coupled stages: *exposure-aware routing, trace recording, TRACE Score assessment, and self-revision.* Together, these stages form a closed loop in which recorded traces update the routing and verification policies used by later configurations.

Firstly, lst us consider the motivating trace in Figure 1. We can find that: Under all-context sharing, the aggregator receives a supported report, an irrelevant report, and an unsupported report with equal visibility. In TRACE-MAS, exposure-aware routing first turns this undifferentiated context pool into a controlled

report list. Ranking and filtering decide which reports downstream agents may read; trace records store the routing decision; and TRACE Score penalizes configurations that still allow unsupported reuse. If the same failure recurs, the self-revision loop can reduce the context budget, insert a verifier before aggregation, or adjust the scoring weights. Thus, routing is the first intervention, while assessment and self-revision decide whether the intervention should be retained and how future configurations should change.

### *4.1. Problem Formulation: Report Exposure*

Let a MAS configuration be $\mathcal{C} = (\mathcal{V}, \mathcal{G}, \Theta)$, where $\mathcal{V} = \{v_1, \ldots, v_n\}$ is a set of agents, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed communication graph, and $\Theta$ contains prompts, tools, model assignments, execution settings, and assessment settings. For a task $q$, executing $\mathcal{C}$ produces intermediate reports $\mathcal{R}_t = \{r_1, \ldots, r_t\}$ before a downstream agent $v_i$ runs.

**Definition 1** (Report visibility)**.** For an agent $v_i$, a report-selection policy $\pi$ selects a subset $\mathcal{S}_i = \pi(q, v_i, \mathcal{R}_t, \mathcal{G}, B_i)$, where $B_i$ is a context budget such as a top-$k$ report limit or token limit. The set $\mathcal{S}_i$ is the internal evidence visible to $v_i$.

We record which reports are shown with a task-level binary matrix $M_{ij}^{(q)} = \mathbb{I}[r_j \in \mathcal{S}_i]$, and record downstream reliance with $R_{ij}^{(q)} = \mathbb{I}[v_i \text{ cites, copies, or semantically reuses } r_j]$.[1] Operationally, $\mathcal{D}_{\text{reuse}}$ first checks explicit report references, then lexical copying, then semantic reuse, and finally sends ambiguous exposed report-response pairs to the verifier.

Detector uncertainty only enters terms that contain $R_{ij}^{(q)}$, while visibility counts based on $M_{ij}^{(q)}$ remain unaffected. The distinction matters: a report can be exposed without being reused, and it can be filtered before any downstream agent can amplify it. The pair $(M, R)$ lets us separate ranking failures from aggregation failures.

The central design problem is to choose $\mathcal{C}$ and $\pi$ so that visible reports preserve useful collaboration while suppressing unsupported or harmful reports:

$$\begin{aligned}\max_{\mathcal{C},\pi} \quad & \text{Utility}(\mathcal{C}, \pi; q) + \lambda_s \text{Support}(\tau) + \lambda_c \text{Consistency}(\tau) + \lambda_r \text{Robustness}(\tau) \\ & - \lambda_h \text{Risk}(\tau) - \lambda_u \text{Unsupp}(\tau) - \beta \text{Cost}(\tau).\end{aligned} \tag{1}$$

Here $\tau$ denotes the execution trace, including reports, routing decisions, verifier outputs, task outcomes, and cost. This equation denotes the report-selection objective that turns hidden context sharing into a measurable report-selection problem.

### *4.2. TRACE Score*

TRACE-MAS operationalizes Eq. 1 with a modular TRACE Score:

$$\text{TS}(\tau) = \alpha_y y(\tau) + \alpha_s s(\tau) + \alpha_c c(\tau) + \alpha_r r(\tau) - \alpha_h h(\tau) - \alpha_u u(\tau) - \alpha_d d(\tau) - \beta z(\tau), \tag{2}$$

where $y$ is task performance, $s$ is evidence support, $c$ is report consistency, $r$ is robustness under perturbation, $h$ is hallucination risk, $u$ is unsupported-claim rate, $d$ is contradiction penalty, and $z$ is normalized cost. Missing integrity metrics can be omitted, which allows the framework to remain compatible with ordinary accuracy-cost evaluation. In the full setting, however, configurations are selected not only because they answer correctly, but because they do so with supported and stable evidence paths.

For trace-level scoring, we compute the major terms from report labels and routing records rather than from the final answer alone. Let $g_j \in [0, 1]$ be the support label of report $r_j$, $\eta_j \in [0, 1]$ be its risk label, and $\ell_j$ be its token length. We use

$$s(\tau) = \frac{\sum_{i,j} M_{ij}^{(q)} R_{ij}^{(q)} g_j}{\sum_{i,j} M_{ij}^{(q)} R_{ij}^{(q)} + \epsilon}, \qquad h(\tau) = \frac{\sum_{i,j} M_{ij}^{(q)} \eta_j}{\sum_{i,j} M_{ij}^{(q)} + \epsilon}. \tag{3}$$

[1] We use a fixed reuse-detector cascade $\mathcal{D}_{\text{reuse}}$ for all compared methods. The concrete thresholds and related TRACE-MAS hyperparameters are reported in Appendix Appendix B.4.

$$z(\tau) = \frac{\log(1 + \sum_j \ell_j + \ell_{\text{verify}})}{\log(1 + B_{\max})}. \tag{4}$$

The consistency term is computed over report pairs that reach the same downstream decision point:

$$c(\tau) = 1 - \frac{\sum_{(j,k)\in\mathcal{P}_{\text{co}}} \text{contr}(r_j, r_k)}{|\mathcal{P}_{\text{co}}| + \epsilon}, \tag{5}$$

where $\mathcal{P}_{\text{co}}$ contains co-visible report pairs and $\text{contr}(\cdot,\cdot)$ is a verifier or heuristic contradiction detector. These definitions are intentionally modular: if a benchmark only provides final task labels, Eq. 2 can still be evaluated with $y$ and $z$; if trace annotations are available, the integrity terms become active and guide revision.

### *4.3. Exposure-Aware Routing*

For each downstream agent, TRACE-MAS forms a candidate report set $\mathcal{A}_i$. This set may include all previous reports, only direct graph parents, or reports satisfying a role-specific eligibility constraint. Candidate reports are ranked by a score

$$\rho(q, v_i, r_j) = w_1\text{rel}(q, r_j) + w_2\text{prov}(r_j) + w_3\text{conf}(r_j) + w_4\text{hist}(v_j) - w_5\text{risk}(r_j), \tag{6}$$

where rel measures task relevance, prov reflects provenance and topology validity, conf captures explicit confidence or verifier support, hist captures historical source reliability, and risk penalizes unsupported or contradictory content. The reports shown to the downstream agent are

$$\mathcal{S}_i = \text{TopK}_{r_j\in\mathcal{A}_i}\rho(q, v_i, r_j). \tag{7}$$

More explicitly, the routed set is constrained by both rank and risk:

$$\mathcal{S}_i = \{r_j \in \mathcal{A}_i : \text{rank}_{\rho,i}(r_j) \le k_i, \quad \rho(q, v_i, r_j) \ge \gamma_i, \quad \text{risk}(r_j) \le \delta_i\}. \tag{8}$$

When the system needs a soft routing signal for revision analysis, we use a temperature-scaled visibility probability

$$p_i(r_j) = \frac{\exp(\rho(q, v_i, r_j)/\tau_r)}{\sum_{r_m\in\mathcal{A}_i}\exp(\rho(q, v_i, r_m)/\tau_r)} \cdot \mathbb{I}[\text{risk}(r_j) \le \delta_i]. \tag{9}$$

The hard top-$k$ decision in Eq. 8 is used during execution, while $p_i(r_j)$ gives a diagnostic view of which reports are close to the routing boundary. In the prototype, lexical, BM25, and embedding rankers instantiate rel, while verifier outputs instantiate risk and confidence terms. So, stronger LLM rerankers or late-interaction models can replace the lightweight ranker when the budget permits.

### *4.4. Trace Records*

Each report is stored with records for the source agent, role, model, prompt type, rank score, support label, visibility decision, downstream visibility, and filtering reason. Formally, a trace is represented as

$$\tau_q = \big(q, \mathcal{C}, \{r_j\}_{j=1}^t, M^{(q)}, R^{(q)}, \{g_j, \eta_j, \rho_j\}_{j=1}^t, a_q, y_q, z_q\big), \tag{10}$$

where $a_q$ is the final answer or action sequence. These records enable two operations. First, the verifier can inspect where a report came from rather than only the report text. Second, the self-revision loop can diagnose whether failures came from too little sharing, too much sharing, poor ranking, weak verification, or an inappropriate scoring profile.

**Algorithm 1** Self-Revision Loop

**Require:** Initial configuration $\mathcal{C}_0$, task set $\mathcal{Q}$, revision budget $T$, operator set $\mathcal{O}$, context budgets $\{B_i\}$
**Ensure:** Best configuration $\mathcal{C}^\star$ and audited trace records

$\mathcal{C}^\star \leftarrow \mathcal{C}_0$, best $\leftarrow -\infty$
**for** $t = 0$ to $T-1$ **do**
  **for all** $q \in \mathcal{Q}$ **do**
    Execute $\mathcal{C}_t$; collect candidate reports $\mathcal{A}_i(q)$ for each downstream agent $v_i$
    Score reports with Eq. 6; expose $\mathcal{S}_i$ by Eq. 8
    Run verifier checkpoints when triggered; store $M^{(q)}$, $R^{(q)}$, support labels, risk labels, and costs
  **end for**
  Compute mean TRACE Score $\overline{\mathrm{TS}}_t$ and diagnostic vector $\mathbf{d}_t$ using Eqs. 2–11
  Identify failure modes: noisy visibility, unsupported reuse, contradiction propagation, false consensus, or excessive cost
  **for all** $o \in \mathcal{O}$ **do**
    Generate candidate $\mathcal{C}_t^o \leftarrow o(\mathcal{C}_t, \mathbf{d}_t)$ by a bounded routing, verifier, budget, ranker, or scoring revision
    Re-evaluate $\mathcal{C}_t^o$ under the same task subset, backbone, and budget protocol
    Compute penalized improvement $\Delta(o)$ by Eq. 12
  **end for**
  $o^\star \leftarrow \arg\max_{o \in \mathcal{O}} \Delta(o)$
  **if** $\Delta(o^\star) > \epsilon_{\mathrm{rev}}$ **then**
    $\mathcal{C}_{t+1} \leftarrow \mathcal{C}_t^{o^\star}$
  **else**
    $\mathcal{C}_{t+1} \leftarrow \mathcal{C}_t$
  **end if**
  **if** $\overline{\mathrm{TS}}_t >$ best **then**
    $\mathcal{C}^\star \leftarrow \mathcal{C}_t$, best $\leftarrow \overline{\mathrm{TS}}_t$
  **end if**
**end for**
**return** $\mathcal{C}^\star$

### *4.5. Self-Revision Loop*

The self-revision loop follows the sequence observe, diagnose, revise, and reassess. The revisions are bounded and targeted: they modify the configuration only when recorded traces indicate a concrete report-sharing failure. This design keeps the method focused on auditable repair rather than unconstrained configuration search.

At round $t$, we aggregate trace diagnostics into

$$\mathbf{d}_t = [\mathrm{UCR}_t, \mathrm{NER}_t, \mathrm{HCR}_t, \mathrm{FCR}_t, 1 - \mathrm{EPrec}_t, z_t], \tag{11}$$

where the entries denote unsupported-claim rate, noisy-exposure rate, hallucination-cascade rate, false-consensus rate, evidence-imprecision, and normalized cost. A revision operator $o \in \mathcal{O}$ maps $(\mathcal{C}_t, \mathbf{d}_t)$ to a candidate configuration $\mathcal{C}_t^o$. We accept the best candidate according to the penalized improvement

$$\Delta(o) = \mathrm{TS}(\tau(\mathcal{C}_t^o)) - \mathrm{TS}(\tau(\mathcal{C}_t)) - \lambda_z \max(0, z(\mathcal{C}_t^o) - z(\mathcal{C}_t) - \xi_z), \tag{12}$$

with acceptance condition $\Delta(o) > \epsilon_{\mathrm{rev}}$. This makes revision a trace-supported repair decision: an operator must improve the score enough to justify any extra verification or routing cost.

The revision operators are summarized in Table 2. They are designed to be interpretable: when a revision is accepted, the trace states which report-sharing problem it addressed.

### *4.6. Prompt Construction*

In our TRACE-MAS, the prompt templates are deliberately role-bounded. Worker prompts define the local task-solving interface, aggregation prompts require a consolidated answer from routed reports, verifier prompts inspect support, contradiction, overconfidence, and recommended report weights, and judge prompts select useful processor responses under a compact output contract. This design keeps natural-language instructions aligned with the formal trace fields above: task text maps to $q$, visible reports map to $\mathcal{S}_i$, routing records map to visibility decisions, and verifier output contributes to support and risk labels. We report the key prompt templates in Appendix C. The templates include the execution prompt for worker agents (Appendix C.1), the aggregation prompt used after report routing (Appendix C.2), the verifier prompt inserted by verifier-revision operators (Appendix C.3), and the judge prompt used in SMoA-style selection (Appendix C.4). We omit benchmark-specific task text and API boilerplate, since those fields are supplied at runtime and do not change the method-level prompt contract.

| Operator | Trace trigger | Configuration change |
|---|---|---|
| Routing revision | Noisy or unsupported reports are visible to downstream agents | Switch all-context or direct-parent routing to ranked top-$k$ or risk-aware routing |
| Exposure-budget revision | Useful reports are filtered, or too many risky reports remain visible | Adjust top-$k$ or token budget by agent role |
| Verifier insertion | Unsupported claims reach aggregation or final answer | Insert an integrity checkpoint before aggregator or high-risk sink node |
| Ranker update | Relevance ranking fails to separate supported and risky reports | Change lexical, BM25, embedding, or LLM-reranking policy |
| Assessment-profile update | Accuracy improves while integrity metrics degrade | Increase weights on evidence support, hallucination risk, contradiction, or cost |

Table 2: Self-revision operators. Each operator changes a bounded part of the configuration and is triggered by a recorded diagnosis.

### *4.7. Theoretical Intuition: Visibility-Risk Reduction*

We use a monotonicity intuition to explain why routing can reduce harmful reuse when the ranker gives safer reports higher scores. This analysis is not meant to replace empirical validation. Instead, it connects the routing rule in Eq. 8, the trace record in Eq. 10, and the risk term in Eq. 3.

**Assumption 1** (Risk-aware ordering)**.** For a candidate set $\mathcal{A}_i$, the routing score $\rho$ is risk-aware if, for any risky report $r^-$ and supported report $r^+$ with similar task relevance, $\rho(q, v_i, r^+) > \rho(q, v_i, r^-)$ with probability at least $1 - \epsilon$.

For an agent $v_i$, define expected visibility risk as

$$\mathcal{E}_i(\pi) = \sum_{r \in \mathcal{A}_i} \Pr_{\pi}(r \in \mathcal{S}_i) \cdot p_i^{\text{reuse}}(r) \cdot \eta(r), \tag{13}$$

where $p_i^{\text{reuse}}(r)$ is the probability that $v_i$ reuses report $r$ once it is visible and $\eta(r) \in [0, 1]$ is the report risk level. All-context sharing sets $\Pr(r \in \mathcal{S}_i) = 1$ for every candidate report. Top-$k$ routing instead assigns high visibility probability to reports with larger $\rho$.

*Intuition.* If the ranker gives safer reports higher scores and report reuse is nondecreasing in visibility, replacing all-context sharing with the gated top-$k$ policy in Eq. 8 should reduce $\mathcal{E}_i$ whenever at least one risky report is excluded and at least one supported report remains visible. Under all-context sharing, every candidate report has visibility probability one, so every risky report contributes $p_i^{\text{reuse}}(r)\eta(r)$ to Eq. 13. Under gated top-$k$ routing, reports outside $\mathcal{S}_i$ have zero direct visibility and therefore lose their direct harmful-reuse contribution. The risk-aware ordering assumption places supported reports above similarly relevant risky reports with probability at least $1-\epsilon$. The gate then removes reports that fall below the rank, score, or risk thresholds before downstream reuse can occur. When at least one risky report is excluded and one supported report remains visible, the harmful-reuse sum decreases while useful evidence remains available, provided the excluded risky report has nonzero reuse probability and positive risk.

This intuition explains why routing and verification are complementary. Routing reduces the chance that harmful reports are seen; verification reduces the chance that a remaining report is treated as supported. The TRACE Score then encourages the self-revision loop to preserve changes that improve this tradeoff instead of changes that only increase raw context volume.

## 5. Experiments

We evaluate whether explicit report-visibility control improves both task performance and integrity. The experiments answer six research questions.

- **RQ1:** Does TRACE-MAS remain competitive with existing MAS methods on standard benchmarks?
- **RQ2:** Does exposure-aware routing reduce unsupported claims and noisy reports reaching downstream agents?
- **RQ3:** Does TRACE Score select more trustworthy configurations than task-only assessment?
- **RQ4:** Which self-revision operators contribute most?
- **RQ5:** What is the cost-integrity tradeoff?
- **RQ6:** Do the gains persist under transfer and stronger backbone models?

### *5.1. Experimental Setup*

**Benchmarks.** We use Big-Bench Extra Hard (BBEH) and BBEH-Mini for general reasoning, WorkBench for workplace tool use, and SWE-Bench-Lite and SWE-Bench-Verified for software engineering [30, 31, 32]. BBEH-Mini contains 460 examples, WorkBench contains 690 tasks, SWE-Bench-Lite contains 300 issues, and SWE-Bench-Verified contains 500 human-validated issues. Appendix Appendix B summarizes the benchmark and backbone settings. To directly test risky report propagation, we construct noisy variants by injecting irrelevant, contradictory, fluent unsupported, partially correct, or mixed noisy reports into intermediate agent context. These settings model the information-access situation that motivates this paper: downstream agents must decide which retrieved evidence and intermediate reports deserve visibility.

**Baselines.** Standard baselines include Direct LLM Call, Single Agent, Majority Vote, Peer-review and debate-style MAS [9, 10], software-oriented MAS frameworks [7, 8], automatic MAS generators [11, 12, 13], and configuration or workflow search methods [14, 15, 23]. We also include recent collaborative MAS baselines in the standard-results table when their reported settings match our benchmark groups [33, 34]. Diagnostic exposure baselines include all-context routing, direct-parent routing, random top-$k$, ranked top-$k$, and ranked top-$k$ plus verifier. We keep these diagnostic variants outside the main standard-results table because they are module isolations rather than complete systems.

**Backbones.** We report fixed-backbone results for Claude-3.5-Sonnet (C-3.5S), Qwen3-235B-A22B (Q-235B), and Qwen3-Coder-480B-A35B (Q-480B), plus LLM-selection results where the system assigns models to roles. We also run a strong-backbone analysis with GPT-5.5 and Claude Opus 4.6 to test whether model scaling removes the need to control report visibility. Additional backbone details are summarized in Appendix Appendix B.2 (API access, temperature, pricing, and reproducibility notes are in Appendix Appendix B.3).

**Metrics.** For task performance, we report accuracy on reasoning/tool-use tasks and issue resolution rate on SWE-Bench. For integrity, we report unsupported-claim rate, hallucination cascade rate, false consensus rate, noisy-report exposure, evidence precision, TRACE Score, and token cost. Unless otherwise stated, each task query is run for three independent trials and reported results are averaged.

### *5.2. RQ1: Main Results*

The first experiment asks whether TRACE-MAS remains competitive in the standard MAS evaluation setting before we isolate its integrity behavior. This is important because report filtering could, in principle, make agents overly conservative: if the system hides too many intermediate reports, downstream agents may lose useful cross-branch evidence and standard task accuracy may decrease. Table 3 follows the EvoMAS-style standard-results layout and keeps the backbone dimension explicit. We report each method under BBEH, WorkBench, SWE-Bench-Lite, and SWE-Bench-Verified, with separate columns for Claude-3.5-Sonnet (C-3.5S), Qwen3-235B-A22B (Q-235B), and Qwen3-Coder-480B-A35B (Q-480B) whenever the method is evaluated with a fixed single backbone. LLM-selection rows summarize automatic role-level model assignment rather than averaging fixed-backbone results.

The main result is that TRACE-MAS improves standard task performance while adding routing and trace-based assessment. Under fixed backbones, TRACE-MAS improves over EvoMAS per-LLM on every reported dataset-backbone pair. On BBEH, the gains are 45.8 vs. 44.5 with C-3.5S, 55.2 vs. 52.8 with Q-235B, and 45.2 vs. 43.0 with Q-480B. On SWE-Bench-Verified, where the target metric is issue resolution rate rather than multiple-choice accuracy, TRACE-MAS also improves the fixed-backbone results from 42.7

to 45.0 with C-3.5S, from 54.2 to 56.5 with Q-235B, and from 60.4 to 61.9 with Q-480B. These improvements are not confined to one task family: they appear in general reasoning, workplace tool use, and software repair.

The LLM-selection setting provides a stronger test because the system must not only construct a useful collaboration pattern but also assign appropriate models to different roles. TRACE-MAS improves over EvoMAS LLM-selection on all four headline benchmarks: BBEH (59.3 vs. 58.7), WorkBench (50.8 vs. 48.9), SWE-Bench-Lite (55.8 vs. 52.9), and SWE-Bench-Verified (65.0 vs. 63.8). More importantly, Table 3 separates the standard comparison from diagnostic module variants: it answers the headline performance question, while the following RQs explain whether the gains correspond to improved evidence integrity rather than more aggressive context sharing.

| Category | Method | BBEH (%) | | | WorkBench (%) | | | SWE-Lite (%) | | | SWE-Verified (%) | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | C-3.5S | Q-235B | Q-480B | C-3.5S | Q-235B | Q-480B | C-3.5S | Q-235B | Q-480B | C-3.5S | Q-235B | Q-480B |
| Direct | Direct LLM Call | 20.3 | 15.1 | 11.4 | 17.0 | 15.8 | 13.7 | – | – | – | – | – | – |
| Single | Single Agent | 33.2 | 37.8 | 27.4 | 40.1 | 35.6 | 30.1 | 23.0 | 36.3 | 40.8 | 33.6 | 50.2 | 56.4 |
| Predefined MAS | MetaGPT [8] | – | – | – | – | – | – | 29.7 | 42.9 | 39.7 | 40.9 | 44.3 | 43.8 |
| | ChatDev [7] | – | – | – | – | – | – | 20.3 | 32.0 | 36.0 | 26.1 | 31.2 | 36.2 |
| | Peer Review [10] | 38.4 | 46.2 | 36.0 | 32.3 | 30.8 | 28.5 | 26.0 | 41.0 | 46.1 | 35.6 | 39.4 | 41.2 |
| | Croto [33] | 33.6 | 41.1 | 29.7 | 35.0 | 28.3 | 25.1 | – | – | – | – | – | – |
| | SMoA [34] | 37.3 | 43.9 | 34.6 | 29.9 | 30.6 | 22.3 | – | – | – | – | – | – |
| | Majority Vote | 36.4 | 44.3 | 32.2 | 39.3 | 36.4 | 33.3 | 24.1 | 38.0 | 42.8 | 30.3 | 33.8 | 36.5 |
| | Multi-Agent Debate [9] | 38.2 | 45.0 | 34.4 | 33.2 | 29.2 | 25.7 | 16.3 | 25.7 | 28.9 | 28.9 | 31.5 | 33.7 |
| MAS Generation | AutoAgents [11] | 23.8 | 30.2 | 23.4 | 40.4 | 37.8 | 34.6 | 3.6 | 14.7 | 16.4 | 8.7 | 26.4 | 28.8 |
| | MAS-GPT [12] | 18.9 | 12.1 | 18.2 | 14.4 | 18.1 | 20.6 | 0.0 | 2.3 | 4.1 | 3.1 | 7.6 | 9.4 |
| | ADAS [13] | 22.7 | 32.4 | 28.6 | 39.2 | 33.1 | 31.2 | 17.6 | 27.8 | 31.2 | 29.9 | 34.6 | 39.1 |
| MAS Evolution | EvoAgent [23] | 48.2 | 43.5 | 36.7 | 41.8 | 36.2 | 33.4 | 24.6 | 38.8 | 43.6 | 36.1 | 51.8 | 53.3 |
| | EvoMAS [15] (per-LLM) | 44.5 | 52.8 | 43.0 | 44.5 | 39.7 | 35.0 | 33.9 | 48.2 | 57.6 | 42.7 | 54.2 | 60.4 |
| | EvoMAS [15] (LLM-selection) | | **58.7** | | | 48.9 | | | 52.9 | | | 63.8 | |
| | TRACE-MAS (per-LLM) | 45.8 | 55.2 | 45.2 | 46.6 | 41.4 | 36.2 | 34.2 | 50.8 | **60.8** | 45.0 | 56.5 | 61.9 |
| | TRACE-MAS (LLM-selection) | | **59.3** | | | **50.8** | | | 55.8 | | | **65.0** | |

Table 3: Main results across all benchmarks. We report task accuracy (%) for BBEH and WorkBench, and issue resolution rate (%) for SWE-Bench-Lite and SWE-Bench-Verified. Results are shown separately for three fixed backbones: Claude-3.5-Sonnet (C-3.5S), Qwen3-235B-A22B (Q-235B), and Qwen3-Coder-480B-A35B (Q-480B). LLM-selection rows use automatic role-level model assignment. Values for TRACE-MAS are averaged over three independent runs.

### *5.3. RQ2: Exposure Routing and Hallucination Cascades*

The routing ablation tests the central premise of the paper: in a multi-agent system, the question is not only what each agent writes, but also which reports become visible to later agents. We compare five report-sharing regimes. All-context routing gives every downstream agent access to every prior report. Direct-parent routing shows only reports along the local dependency path. Random top-$k$ limits context size without ranking. Ranked top-$k$ selects reports by relevance. TRACE-MAS adds verification and trace-based scoring on top of ranked routing.

All-context sharing is fragile under noisy reports. On Noisy-BBEH-Mini, all-context obtains 41.57% accuracy with 36.8% unsupported claims and 29.4% noisy exposure. Direct-parent routing reduces visibility risk to 17.2% unsupported claims and 13.6% noisy exposure, but it can discard useful evidence from sibling branches. Random top-$k$ shows why context reduction alone is insufficient: with $k = 2$, it uses fewer tokens than all-context routing but still underperforms ranked routing in both accuracy and integrity. Ranked top-$k$ improves the tradeoff because it selects semantically useful reports, and the embedding ranker reaches 45.43% accuracy with 14.2% unsupported claims. The full routing plus verifier variant reaches 47.53% accuracy, 9.2% unsupported claims, and 7.0% noisy exposure.

The same pattern appears on Noisy-WorkBench, where reports often mix natural-language reasoning with tool-state facts. All-context routing reaches only 38.17% accuracy and 34.5% unsupported claims, while ranked top-$k$ improves accuracy to 41.03% and reduces unsupported claims to 15.8%. Adding routing and verification further raises accuracy to 44.53% and reduces noisy exposure to 7.8%. WorkBench errors are often caused by plausible but wrong references to emails, task boards, or calendar states. A system that merely shortens context may still expose the wrong tool-state summary; TRACE-MAS instead changes the visibility rule so that downstream agents see reports with stronger evidence support.

| Setting | Routing policy | Acc. | UCR | NER | EPrec |
|---|---|---|---|---|---|
| *BBEH-Mini clean, C-3.5S* | | | | | |
| Clean | all-context | 49.07 | 19.4 | – | 71.6 |
| Clean | direct-parent | 48.17 | 10.8 | – | 83.1 |
| Clean | embedding top-$k$, $k = 2$ | 47.13 | 7.2 | – | 82.8 |
| Clean | TRACE-MAS routing + verifier | **49.43** | **5.2** | – | **86.2** |
| *Noisy-BBEH-Mini, C-3.5S* | | | | | |
| Noisy | all-context | 41.57 | 36.8 | 29.4 | 50.8 |
| Noisy | direct-parent | 44.17 | 17.2 | 13.6 | 66.4 |
| Noisy | random top-$k$, $k = 2$ | 40.43 | 22.4 | 17.2 | 57.6 |
| Noisy | embedding top-$k$, $k = 2$ | 45.43 | 14.2 | 10.8 | 70.5 |
| Noisy | TRACE-MAS routing + verifier | **47.53** | **9.2** | **7.0** | **76.8** |
| *Noisy-WorkBench, C-3.5S* | | | | | |
| Noisy | all-context | 38.17 | 34.5 | 28.6 | 51.8 |
| Noisy | embedding top-$k$, $k = 2$ | 41.03 | 15.8 | 12.4 | 68.5 |
| Noisy | TRACE-MAS routing + verifier | **44.53** | **10.2** | **7.8** | **74.5** |

Table 4: Routing-policy ablation. Each row reports mean accuracy, unsupported-claim rate (UCR), noisy-report exposure (NER), and evidence precision (EPrec) over three independent runs when applicable. Exposure-aware routing with verification reduces unsupported claims and noisy exposure without relying on larger context.

The cascade stress test makes the mechanism more explicit. As shown in Table 5, fluent unsupported reports are especially dangerous because they look like valid reasoning while lacking evidence. Under 30% fluent unsupported noise, Vanilla all-context routing reaches a hallucination cascade rate of 64.8% and false consensus rate of 42.3%. TRACE-MAS Full reduces these to 18.5% and 10.2%, respectively, while improving accuracy from 39.77% to 47.03%. The reduction is consistent across noise types: irrelevant distractors are less damaging, contradictory reports are more damaging, and fluent unsupported reports are the hardest case, but TRACE-MAS reduces cascade and false-consensus metrics in all of them.

The injection-position analysis further clarifies why all-context sharing is risky. False verifier feedback and intermediate aggregator summaries are treated as authoritative by downstream agents, so their errors propagate more strongly than ordinary worker errors. In the verifier-feedback setting, Vanilla all-context reaches a 71.5% cascade rate and 48.6% false consensus rate, whereas TRACE-MAS reduces them to 20.8% and 12.4%. The benefit is therefore not limited to hiding noisy worker outputs; TRACE-MAS also reduces the chance that verifier or aggregator reports become uncritically reused evidence.

| Noise / stress setting | Vanilla all-context | | | TRACE-MAS Full | | | Δ HCR |
|---|---|---|---|---|---|---|---|
| | Acc. | HCR | FCR | Acc. | HCR | FCR | |
| Irrelevant distractor | 44.70 | 34.2 | 14.5 | 48.46 | 10.8 | 4.8 | -23.4 |
| Contradictory report | 42.43 | 52.6 | 28.5 | 47.84 | 13.4 | 6.2 | -39.2 |
| Fluent unsupported | 39.77 | 64.8 | 42.3 | 47.04 | 18.5 | 10.2 | -46.3 |
| Partially correct | 41.16 | 58.5 | 34.8 | 47.25 | 15.8 | 8.5 | -42.7 |
| Verifier-feedback injection | 38.14 | 71.5 | 48.6 | 46.53 | 20.8 | 12.4 | -50.7 |
| 50% fluent noise | 34.49 | 72.5 | 54.6 | 44.58 | 22.8 | 14.5 | -49.7 |
| Noisy-WorkBench partial | 35.11 | 56.8 | 38.5 | 43.83 | 16.2 | 8.5 | -40.6 |

Table 5: Hallucination-cascade stress test. HCR is hallucination cascade rate and FCR is false consensus rate. The table compares vanilla all-context sharing with full TRACE-MAS under representative noise and stress settings.

To localize where harmful reports are suppressed, we analyze transition dynamics from injection to final answer use. As visualized in Figure 4, out of 100 fluent unsupported reports, Vanilla all-context leaves 85 visible, 63 reused, 48 aggregated, and 42 finalized. TRACE-MAS Full reduces these counts to 14, 6, 3, and 2. This transition-level view is useful because task accuracy alone does not show where an error is stopped.

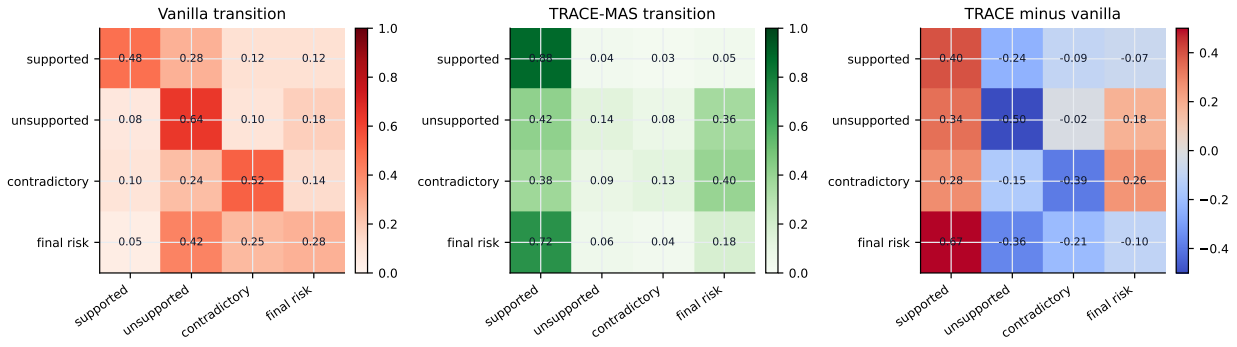



Figure 4: Cascade transition dynamics. TRACE-MAS changes harmful-report propagation from reuse-heavy paths to early blocking paths.

The dominant path shifts from injected → visible → reused → finalized to injected → blocked, showing that the system intervenes before downstream consensus formation. The transition analysis connects the routing module to the final-answer integrity metrics.

### *5.4. RQ3: TRACE Score Assessment*

To evaluate whether trustworthy MAS construction requires a different selection signal, we compare several configuration-scoring objectives in Table 6. This experiment asks whether the search objective should reward only task success or whether it must also encode evidence support, consistency, robustness, hallucination risk, and cost. The difference is visible even in the clean setting: accuracy-only selection reaches the best clean BBEH-Mini accuracy in the table, but it keeps a high unsupported-claim rate because all-context configurations are rewarded whenever the final answer is correct. Full TRACE Score gives up little clean accuracy while reducing unsupported claims from 20.5% to 7.8%.

The difference becomes much larger under noise. On Noisy-BBEH-Mini with C-3.5S, accuracy-only reaches 43.77% accuracy with 32.5% unsupported claims and a 5.8-point robustness drop. Full TRACE Score reaches 47.83% accuracy, 8.5% unsupported claims, and only a 0.4-point robustness drop. On Noisy-WorkBench, the same comparison is 39.47% accuracy and 30.8% unsupported claims for accuracy-only selection, versus 44.23% accuracy and 9.2% unsupported claims for full TRACE Score. These results indicate that the integrity objective changes which configurations are selected, rather than serving only as a post-hoc penalty for unsafe outputs.

The ablation rows identify which part of the score matters. A score without the verifier already improves over accuracy-only because ranked routing reduces visible noise. However, removing the hallucination penalty leaves unsupported claims high: on Noisy-BBEH-Mini, the unsupported-claim rate rises to 22.8% even though accuracy remains 46.53%. This means that accuracy and integrity are not interchangeable optimization targets. TRACE Score works because it separates task correctness from the quality of the evidence path that produced the answer.

| Setting | Selection signal | Acc. | UCR | Robust drop | TS | Selected routing |
|---|---|---|---|---|---|---|
| *BBEH-Mini clean, C-3.5S* | | | | | | |
| Clean | accuracy only | **49.47** | 20.5 | – | 72.4 | all-context large pool |
| Clean | TRACE Score w/o verifier | 48.53 | 12.4 | – | 78.2 | ranked top-$k$ |
| Clean | full TRACE Score | 48.23 | **7.8** | – | **82.4** | ranked + verifier |
| *Noisy-BBEH-Mini, C-3.5S* | | | | | | |
| Noisy | accuracy only | 43.77 | 32.5 | -5.8 | 58.2 | all-context large pool |
| Noisy | TRACE Score w/o verifier | 45.83 | 18.5 | -2.7 | 70.5 | ranked top-$k$ |
| Noisy | TRACE Score w/o hallucination | 46.53 | 22.8 | -2.3 | 68.2 | ranked BM25 larger |
| Noisy | full TRACE Score | **47.83** | **8.5** | **-0.4** | **79.8** | ranked + verifier |
| *Noisy-WorkBench, C-3.5S* | | | | | | |
| Noisy | accuracy only | 39.47 | 30.8 | -6.7 | 56.2 | all-context large pool |
| Noisy | TRACE Score w/o verifier | 42.53 | 17.2 | -3.0 | 68.5 | ranked top-$k$ |
| Noisy | full TRACE Score | **44.23** | **9.2** | **-1.0** | **78.5** | ranked + verifier |

Table 6: TRACE Score ablation. We report mean accuracy, unsupported-claim rate (UCR), robustness drop under noise, scalar TRACE Score (TS), and the selected routing style. Full TRACE Score sacrifices little clean accuracy but strongly reduces unsupported claims and robustness loss.

### *5.5. RQ4: Self-Revision*

The self-revision experiment tests whether configuration repair improves reliability over repeated rounds. The revision loop treats each accepted change as a bounded repair for an observed failure mode: routing can be revised, verifier placement can be inserted, context budgets can be adjusted, score weights can be recalibrated, and verifier thresholds can be changed. Table 7 shows that routing revision is the strongest single operator, while verifier insertion substantially reduces unsupported claims. Routing-only revision raises Noisy-BBEH-Mini accuracy from 41.57% to 45.53% after three rounds and reduces noisy exposure from 29.4% to 13.5%. Verifier-only revision is less effective for accuracy, but it directly reduces unsupported claims.

The operators are complementary. Routing plus verifier reaches 47.23% accuracy and 13.2% unsupported claims at three rounds, while full TRACE-MAS reaches 47.83% accuracy and 8.5% unsupported claims. The raw-accuracy gap is modest, but the integrity gap is large, which suggests that the full revision loop mainly improves the evidence path rather than only the final label. The Noisy-WorkBench rows show a similar pattern: no revision gives 38.17% accuracy and 34.5% unsupported claims, while full TRACE-MAS reaches 44.23% accuracy and 9.2% unsupported claims. This cross-benchmark consistency supports the view that self-revision is acting on a general report-sharing failure rather than overfitting to BBEH-style reasoning tasks.

The round ablation determines the practical stopping point. The first round gives the largest jump because many configurations begin with all-context sharing or weak filtering. Rounds two and three continue to reduce unsupported claims, but rounds four and five produce diminishing returns: from round three to round five, accuracy increases only from 47.83% to 48.03%, while token use increases from 1356K to 1548K. We therefore use three rounds as the default practical budget. This choice is also consistent with the acceptance-rate pattern: accepted revisions drop from 65.5% in round one to 52.8% in round three and 38.5% in round five, indicating that most high-value repairs have already been applied by the third round.

| Setting | Revision setting | Rounds | Acc. | UCR | NER | Accept |
|---|---|---|---|---|---|---|
| *Noisy-BBEH-Mini, C-3.5S* | | | | | | |
| Noisy-BBEH | no revision | 0 | 41.57 | 36.8 | 29.4 | – |
| Noisy-BBEH | routing only | 3 | 45.50 | 21.8 | 13.5 | 54.2 |
| Noisy-BBEH | verifier insertion only | 3 | 44.55 | 22.4 | 24.5 | 50.2 |
| Noisy-BBEH | routing + verifier | 3 | 47.22 | 13.2 | 9.8 | 52.5 |
| Noisy-BBEH | full TRACE-MAS | 3 | 47.81 | 8.5 | 7.0 | 52.8 |
| Noisy-BBEH | full TRACE-MAS | 5 | **48.08** | **7.8** | **6.5** | 38.5 |
| *Noisy-WorkBench, C-3.5S* | | | | | | |
| Noisy-WorkBench | no revision | 0 | 38.17 | 34.5 | 28.6 | – |
| Noisy-WorkBench | routing only | 3 | 41.83 | 20.5 | 14.2 | 52.5 |
| Noisy-WorkBench | routing + verifier | 3 | 43.53 | 11.2 | 10.5 | 50.5 |
| Noisy-WorkBench | full TRACE-MAS | 3 | **44.23** | **9.2** | **8.0** | 51.2 |

Table 7: Self-revision ablation. Results are averaged over three independent runs. The table keeps the operator groups and revision rounds needed for the main argument.

To inspect how the revision loop changes configurations, we report operator-level diagnostics in Table 8. Routing revisions are frequent, but they are not the only accepted changes. Verifier insertion improves the average trace score from 68.5 to 78.2 and reduces unsupported claims by 12.5 points when accepted. Routing revision produces the largest average unsupported-claim reduction, while verifier-threshold adjustment and $k$-value optimization contribute smaller but still meaningful improvements. The self-revision loop therefore maps observed failures to bounded repairs and accepts a repair only when the score improves. The accepted changes are interpretable configuration repairs, not opaque parameter updates.

| Operator | R1 accept | R2 accept | R3 accept | Before score | After score |
|---|---|---|---|---|---|
| Routing revision | 68.5 | 56.2 | 48.5 | 62.4 | 74.8 |
| Verifier insertion | 64.2 | 52.8 | 45.8 | 68.5 | 78.2 |
| Exposure-budget revision | 58.5 | 46.5 | 40.2 | 70.2 | 74.5 |
| TRACE Score optimization | 52.8 | 42.5 | 36.8 | 72.8 | 76.2 |
| Verifier-threshold adjustment | 60.5 | 50.2 | 43.5 | 71.5 | 77.5 |
| $k$-value optimization | 62.8 | 51.5 | 44.2 | 68.8 | 73.5 |

Table 8: Operator-level revision statistics on Noisy-BBEH-Mini with C-3.5S. Non-routing operators also change trace scores and accepted configurations.

### *5.6. RQ5: Cost-Integrity Tradeoff*

We also study whether the gain comes from better report selection or from more computation. Table 9 varies three cost-relevant factors: the context budget $k$, the ranker, and the verifier strength. The all-context baseline spends most of its context budget on broad report sharing, but much of that context carries unsupported or irrelevant reports. On Noisy-BBEH-Mini, all-context reaches only 41.57% accuracy, 36.8% unsupported claims, and 0.018 integrity per 1K tokens. In contrast, $k = 2$ with an embedding ranker and cheap verifier reaches 47.53% accuracy, 8.8% unsupported claims, and 0.054 integrity per 1K tokens.

The grid explains why the default setting uses $k = 2$. A $k = 1$ no-verifier variant has a high integrity-per-token value because it is cheap and filters aggressively, but its accuracy drops to 39.87% on Noisy-BBEH-Mini and 32.53% on Noisy-WorkBench. This setting often misses necessary cross-branch evidence. Increasing to $k = 3$ recovers more evidence but also raises visibility and token cost; on Noisy-BBEH-Mini, the $k = 3$ cheap-verifier setting improves accuracy to 48.50% but increases unsupported claims to 10.8% and lowers integrity efficiency. Moderate report sharing is therefore an accuracy-integrity tradeoff: too little context loses evidence, and too much context reintroduces noisy reports.

Verifier strength has a similar tradeoff. A strong verifier gives the best integrity upper bound: on Noisy-BBEH-Mini, $k = 2$ with a strong verifier reaches 49.53% accuracy and 5.2% unsupported claims. However,

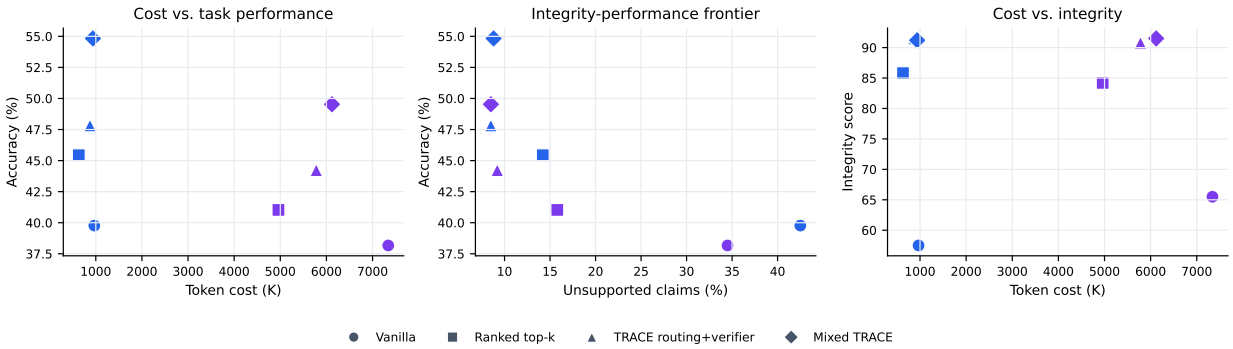



Figure 5: Cost, task performance, and integrity tradeoffs across Noisy-BBEH-Mini and Noisy-WorkBench. Marker shape indicates method and color indicates dataset.

its integrity per 1K tokens is lower than the cheap verifier because verification consumes a larger share of the budget. On Noisy-WorkBench, the cheap verifier reaches 43.53% accuracy and 10.2% unsupported claims, while the strong verifier reaches 45.23% and 6.5% at higher cost. We therefore report the cheap verifier as the default and treat the strong verifier as an integrity-critical option rather than the main efficiency point.

| Setting | Exposure/ranker | Verifier | Acc. | UCR | EPrec | Integrity/1K |
|---|---|---|---|---|---|---|
| *Noisy-BBEH-Mini, C-3.5S* | | | | | | |
| Noisy-BBEH | all-context | none | 41.57 | 36.8 | 50.8 | 0.018 |
| Noisy-BBEH | $k = 1$, embedding | none | 39.87 | 13.8 | 68.5 | **0.054** |
| Noisy-BBEH | $k = 2$, embedding | cheap | 47.53 | 8.8 | 79.8 | **0.054** |
| Noisy-BBEH | $k = 2$, embedding | strong | 49.53 | 5.2 | 84.8 | 0.043 |
| Noisy-BBEH | $k = 3$, embedding | cheap | 48.50 | 10.8 | 81.2 | 0.048 |
| *Noisy-WorkBench, C-3.5S* | | | | | | |
| Noisy-WorkBench | all-context | none | 38.17 | 34.5 | 51.8 | 0.021 |
| Noisy-WorkBench | $k = 2$, embedding | none | 41.03 | 15.8 | 68.5 | 0.048 |
| Noisy-WorkBench | $k = 2$, embedding | cheap | 43.53 | 10.2 | 74.5 | **0.051** |
| Noisy-WorkBench | $k = 2$, embedding | strong | 45.23 | 6.5 | 80.2 | 0.042 |

Table 9: Cost-integrity tradeoff. The table reports mean accuracy, unsupported-claim rate, evidence precision, and integrity per 1K tokens.

To visualize the interaction among task utility, integrity risk, and computational cost, we plot cost, task accuracy, unsupported claims, and integrity score in Figure 5. The TRACE-MAS variants occupy a better frontier than all-context routing: they reduce unsupported claims while maintaining or improving accuracy, often with lower context cost. This does not mean that every TRACE-MAS configuration is cheaper. Some verified configurations deliberately spend additional verifier tokens. The difference is that these additional tokens are targeted at evidence checking, whereas all-context spends tokens on unfiltered report sharing. This distinction explains why the cost-integrity frontier improves even when total token use is comparable.

We further test how sensitive TRACE-MAS is to verifier strength. Table 10 shows that a cheap Q-235B verifier already captures most of the integrity gain, while strong verification provides the best integrity upper bound. Moving from no verifier to a cheap verifier reduces unsupported claims much more than moving from the cheap verifier to the strongest verifier. The remaining gap is concentrated in fluent unsupported and partially correct misleading reports, where linguistic plausibility makes evidence support harder to judge. We use the cheap verifier as the default because it preserves most of the benefit at much lower verifier-token cost, while still leaving a clear upgrade path for settings where integrity is prioritized over efficiency.

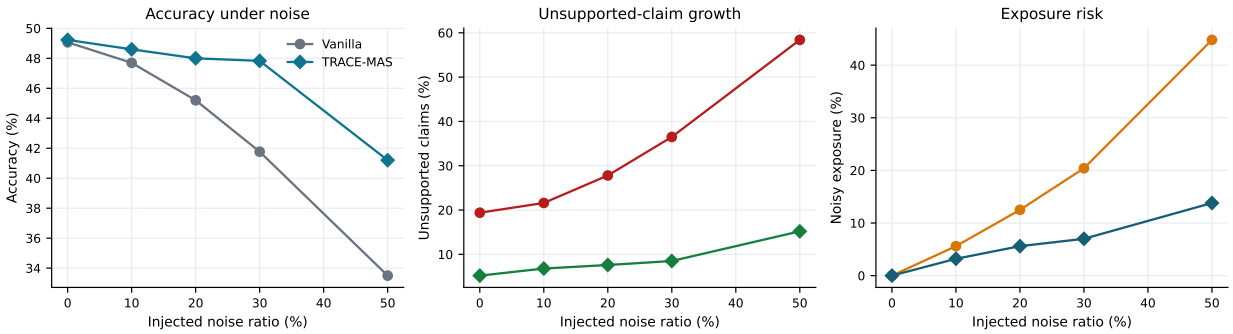



Figure 6: Clean-to-noisy robustness. TRACE-MAS reduces both unsupported-claim growth and noisy exposure as injected noise increases.

| Setting | Verifier | Acc. | UCR | Agree H | FP | FN |
|---|---|---|---|---|---|---|
| *Noisy-BBEH-Mini, C-3.5S* | | | | | | |
| Noisy-BBEH | no verifier | 41.57 | 36.8 | – | – | – |
| Noisy-BBEH | heuristic | 43.58 | 18.8 | 78.5 | 14.2 | 12.5 |
| Noisy-BBEH | cheap Q-235B | 46.50 | 12.5 | 91.2 | 8.5 | 6.8 |
| Noisy-BBEH | cheap C-3.5S | 47.23 | 10.8 | 93.5 | 6.2 | 5.5 |
| Noisy-BBEH | strong C-4S | 47.83 | 8.5 | 96.6 | 3.2 | 3.8 |
| Noisy-BBEH | strong C-4.5S | **48.22** | **7.8** | **97.2** | **2.8** | **3.2** |
| *Noisy-WorkBench, C-3.5S* | | | | | | |
| Noisy-WorkBench | no verifier | 38.17 | 34.5 | – | – | – |
| Noisy-WorkBench | heuristic | 40.53 | 22.5 | 75.5 | – | 15.2 |
| Noisy-WorkBench | cheap Q-235B | 42.83 | 14.8 | 89.8 | – | 8.5 |
| Noisy-WorkBench | strong C-4S | **44.23** | **9.2** | **95.8** | – | **4.5** |

Table 10: Verifier sensitivity. Agree H denotes agreement with human labels, and FP/FN denote false positives and false negatives. Strong verifiers improve integrity most, but cheap LLM verification captures much of the gain.

### *5.7. RQ6: Robustness, Transfer, and Strong Backbones*

To evaluate robustness under increasing contamination, we compare clean-to-noisy degradation in Figure 6. On BBEH-Mini with C-3.5S, Vanilla EvoMAS drops from 49.07% clean accuracy to 28.47% at 50% mixed noise, while TRACE-MAS Full drops from 49.17% to 41.23%. The robustness gap widens as the noise ratio increases: at 10% mixed noise, TRACE-MAS has a small advantage, but at 50% mixed noise the advantage becomes 12.8 points. All-context sharing degrades rapidly as more noisy reports become visible, whereas TRACE-MAS keeps unsupported-claim and noisy-exposure rates bounded through ranked routing and verification.

The robustness analysis by noise type shows that not all noise is equally harmful. Irrelevant distractors are often filtered or ignored even by weaker systems. Contradictory reports are harder because they directly compete with supported evidence. Fluent unsupported reports are the most damaging because they can be reused as if they were a valid reasoning chain. At 30% fluent unsupported noise, Vanilla EvoMAS drops to 39.77% accuracy, while TRACE-MAS remains at 47.03%. The same trend appears on WorkBench, where tool-use reports can contain partially correct but misleading state descriptions. At 50% mixed noise, Vanilla EvoMAS falls to 31.47%, whereas TRACE-MAS retains 41.53%. These results show that the method is not specialized to one type of injected error; it addresses the broader problem of unsupported report visibility.

Report-sharing policies also transfer across related noise settings. Table 11 shows that TRACE-MAS Full optimized on contradiction noise transfers to mixed noise with 88.2% retained accuracy and 14.5% unsupported claims. Policies optimized on mixed-noise 10% transfer to mixed-noise 30% with 89.7% retention, while policies optimized on mixed-noise 50% retain 95.9% when transferred to the milder 30% setting. This

asymmetry suggests that policies learned under severe noise become conservative in a way that remains useful under milder noise.

Cross-benchmark transfer is harder, especially from reasoning tasks to workplace tool-use tasks, but the transferred policies still reduce unsupported claims compared with simpler alternatives. When a BBEH-Mini mixed-noise policy is transferred to Noisy-WorkBench, TRACE-MAS Full retains 75.6% of the direct-optimized accuracy and keeps unsupported claims at 16.8%. Component-transfer diagnostics explain the loss: ranker weights and $k$ values transfer well, while verifier thresholds are more source-specific. This is consistent with the design of TRACE-MAS: relevance ranking captures a general evidence-selection pattern, whereas verification calibration depends more strongly on the form of the task and the noise.

| Source → target | Policy | Direct | Transfer | Retained | UCR |
|---|---|---|---|---|---|
| *Within BBEH-Mini noise transfer.* | | | | | |
| Contradiction 30% → mixed 30% | TRACE-MAS Full | 47.86 | 42.13 | 88.2 | 14.5 |
| Irrelevant 30% → mixed 30% | TRACE-MAS Full | 48.81 | 40.21 | 82.5 | 17.2 |
| Fluent unsupported 30% → mixed 30% | TRACE-MAS Full | 47.01 | 40.80 | 86.8 | 16.2 |
| Mixed 10% → mixed 30% | TRACE-MAS Full | 48.50 | 43.51 | 89.7 | 12.8 |
| Mixed 50% → mixed 30% | TRACE-MAS Full | 41.25 | 39.56 | **95.9** | 18.5 |
| *Cross-benchmark transfer.* | | | | | |
| BBEH mixed 30% → WorkBench mixed 30% | TRACE-MAS Full | 47.03 | 35.51 | 75.6 | 16.8 |
| WorkBench mixed 30% → BBEH mixed 30% | TRACE-MAS Full | 44.26 | 36.50 | 82.6 | 15.8 |

Table 11: Report-sharing policy transfer. The table separates within-BBEH-Mini noise transfer from cross-benchmark transfer. Direct is performance when optimized on the target setting; Transfer is performance after applying a policy learned from the source setting.

Strong-backbone results test whether better individual models remove the need to control report visibility. Table 12 shows that they do not: TRACE-MAS improves over Vanilla EvoMAS by 8.7 points with GPT-5.5 and 9.6 points with Claude Opus 4.6 on Noisy-BBEH-Mini. Strong-pool LLM-selection reaches 70.53% accuracy with 4.2% unsupported claims. The integrity metrics remain important even in this high-capability setting: Vanilla all-context with GPT-5.5 still has 28.5% unsupported claims and 48.5% hallucination cascades, while full TRACE-MAS reduces them to 5.2% and 10.2%.

The strong-backbone experiment also clarifies the relationship between model scaling and system design. Single-agent GPT-5.5 and Claude Opus 4.6 are much stronger than the earlier backbones, but all-context MAS can still underperform when unsupported intermediate reports are widely visible. TRACE-MAS improves the system by controlling propagation, not by assuming that individual agents never err. Table 13 explains the role-level allocation: GPT-5.5 is frequently selected for solver roles, while Claude Opus 4.6 is often selected for verifier, aggregator, and self-revision roles. The mixed strong setting therefore uses model scaling and report selection in a complementary way: stronger models generate better reports, and routing decides which reports should influence later agents.

| Method | Backbone | Noisy-BBEH-Mini | | | | Noisy-WorkBench | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | Acc. | UCR | HCR | NER | Acc. | UCR | HCR | NER |
| Vanilla all-context | GPT-5.5 | 56.53 | 28.5 | 48.5 | 22.5 | 52.53 | 32.5 | 52.5 | 26.8 |
| TRACE-MAS Full | GPT-5.5 | 65.23 | 5.2 | 10.2 | 4.8 | 61.23 | 8.5 | 15.2 | 7.2 |
| Vanilla all-context | Opus 4.6 | 58.23 | 26.8 | 45.2 | 21.2 | 54.23 | 30.8 | 49.8 | 25.2 |
| TRACE-MAS Full | Opus 4.6 | 67.83 | 4.5 | 8.5 | 4.0 | 63.53 | 7.2 | 12.8 | 6.2 |
| TRACE-MAS LLM-selection | Mixed strong | **70.53** | **4.2** | **7.8** | **3.5** | **66.23** | **6.5** | **11.5** | **5.8** |

Table 12: Strong-backbone robustness. Results are organized by method/backbone rows and paired benchmark blocks. Stronger backbones improve raw ability, but all-context sharing still carries high unsupported-claim and cascade rates.

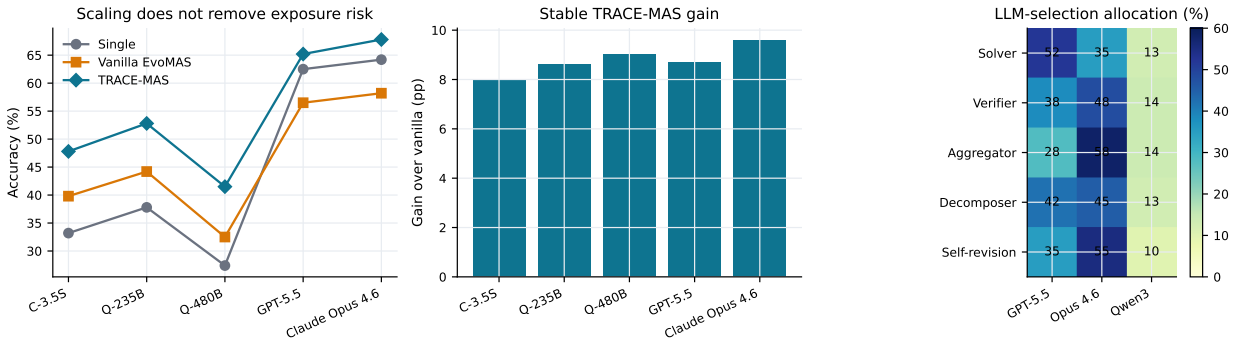



Figure 7: Strong-backbone robustness. Model scaling improves single-agent ability, but report-visibility control still adds stable gains and changes role-level model allocation.

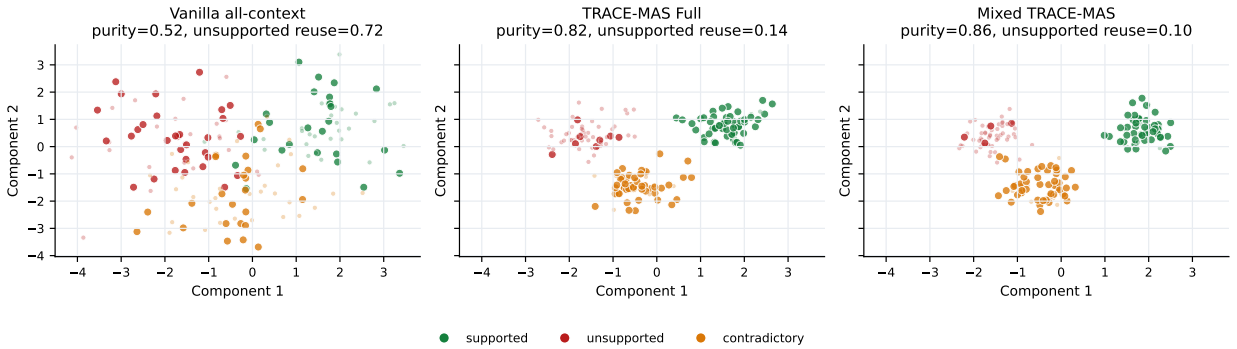



Figure 8: Trace-space projection of intermediate reports. Colors denote report labels, and larger, darker points indicate reports selected for downstream reuse. Compared with all-context routing, TRACE-MAS reduces reuse of unsupported reports while preserving supported evidence.

| Agent role | GPT-5.5 | Opus 4.6 | Q-480B | Mean TS | Task score |
|---|---|---|---|---|---|
| Solver, primary reasoning | 52 | 35 | 13 | 86.5 | 68.5 |
| Solver, secondary reasoning | 48 | 38 | 14 | 85.8 | 67.2 |
| Verifier | 38 | 48 | 14 | 89.2 | 72.5 |
| Aggregator | 28 | 58 | 14 | 88.5 | 71.8 |
| Decomposer | 42 | 45 | 13 | 87.2 | 69.5 |
| Self-revision meta-agent | 35 | 55 | 10 | 90.5 | 74.2 |

Table 13: Role-level model allocation in strong-pool LLM-selection on Noisy-BBEH-Mini. Values are percentages of selected assignments for each role.

### *5.8. Trace-Space and Pareto Analysis*

To further investigate whether the integrity metrics correspond to a measurable structure in the trace distribution, we project report-level trace features into a low-dimensional space in Figure 8. Vanilla all-context mixes supported and unsupported reports, especially under fluent unsupported noise. In the cluster statistics, Vanilla all-context has a silhouette score of 0.18 and cluster purity of 0.52 on mixed-noise Noisy-BBEH-Mini, while TRACE-MAS Full reaches 0.62 and 0.84. The unsupported-cluster reuse rate falls from 0.68 to 0.12, and the supported-cluster routing rate rises from 0.51 to 0.89. Thus, the trace space is not only more separable; the routing decisions are aligned with the supported regions of the space.

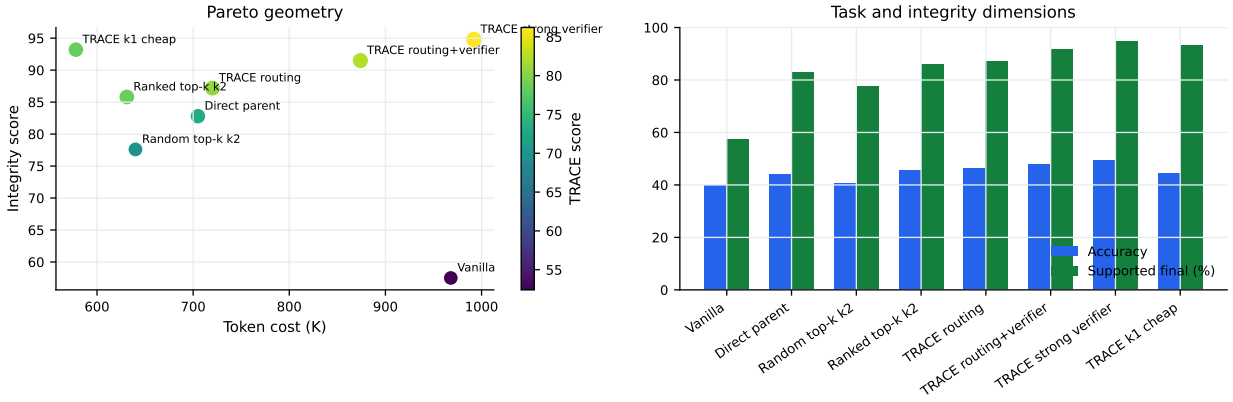



Figure 9: Pareto and metric geometry analysis. TRACE-MAS improves the integrity-performance frontier rather than maximizing raw context size.

The progression across methods explains where the separation comes from. Random top-$k$ slightly improves separation because it reduces context volume, but its unsupported reports are still frequently reused. Ranked top-$k$ produces a larger improvement by incorporating relevance. However, fluent unsupported reports remain difficult because they can be semantically close to supported reports. TRACE-MAS adds verifier and support/contradiction features, which separate fluency from evidence grounding. This is why the improvement is strongest for fluent unsupported reports: linguistic similarity alone is insufficient, but trace features expose the missing support.

To examine whether TRACE Score reflects a useful optimization frontier, we compare utility, integrity, and cost in Figure 9. Some configurations with high accuracy carry high unsupported-claim risk, while TRACE-MAS configurations move toward higher integrity at moderate cost. On Noisy-BBEH-Mini, all-context has 39.77% accuracy, 57.5% integrity, and a TRACE Score of 52.4. TRACE-MAS routing plus verifier reaches 47.53% accuracy, 90.8% integrity, and a TRACE Score of 79.8, while full TRACE-MAS with a strong verifier reaches the highest integrity point at 94.8%. These points demonstrate that the main design goal is not maximum context size or maximum raw accuracy in isolation, but a better frontier over utility, integrity, and cost.

The metric-geometry analysis supports this interpretation. Unsupported-claim rate, noisy-exposure rate, hallucination-cascade rate, and false-consensus rate form a tightly related integrity cluster, while token count is nearly orthogonal to these integrity risks. TRACE Score is negatively correlated with integrity risks and positively correlated with accuracy, which validates it as a scalar proxy for multi-dimensional configuration quality. Together, the visual analyses summarize the empirical mechanism: RQ2 shows that routing changes what agents see, RQ3 shows that the score selects safer configurations, RQ4 shows that self-revision repairs recorded failures, and the trace-space/Pareto views show that these changes reshape how evidence moves through the system.

## 6. Discussion

### *6.1. Method-Level Comparison*

TRACE-MAS uses a configuration-based MAS substrate, so the closest baseline in implementation form is EvoMAS [15]. This relationship should not be confused with the optimization target. EvoMAS optimizes the design of the agent team: roles, prompts, tools, topology, execution settings, and model assignments. Related MAS construction methods similarly emphasize role specialization and workflow generation [6, 11]. Other automated design methods search over agentic programs, topologies, or flow structures [13, 14]. TRACE-MAS adds a layer that operates after the team starts running: which intermediate reports are

shown, which reports are reused, which claims are unsupported, and which recorded failures should trigger a bounded revision.

This difference changes the unit of control. In configuration search, a graph edge usually means that one agent can communicate with another. In TRACE-MAS, the edge only defines an eligible channel. The actual decision is made at the report level by a ranker, a verifier, and a context budget. This design borrows the information-access view that ranking determines which evidence is shown [21, 22], but applies it to agent-generated reports rather than only external documents. This makes it possible to keep useful collaboration while blocking repeated unsupported reports. It also makes the resulting behavior easier to audit because the trace records not only the final answer, but also the visibility, filtering reason, support label, and downstream reuse of each report.

Compared with verifier-only designs, TRACE-MAS intervenes earlier. A final judge can reject a bad answer, but it cannot always reconstruct how an unsupported report shaped intermediate reasoning. This limitation is important because factuality and hallucination evaluators usually inspect claims or answers after they have already been produced [18, 19, 20]. Compared with routing-only designs, TRACE-MAS avoids trusting relevance alone: a report can be relevant and still risky. The experiments support this distinction. Ranking reduces noisy visibility, verification reduces unsupported reuse among visible reports, and TRACE Score decides whether the configuration revision is worth its cost.

### *6.2. Ranking, Verification, and Model Scaling*

The results suggest that trustworthy MAS behavior depends on the interaction between ranking, verification, and model quality. Strong backbones raise the ceiling of individual reasoning, but they do not remove the system-level problem of showing weak reports to later agents. This is consistent with agent benchmarks and tool-use studies showing that stronger LLMs still depend on interaction structure, context management, and feedback signals [1, 2, 3]. In the strong-backbone experiment, GPT-5.5 and Claude Opus 4.6 improve raw ability, while Vanilla EvoMAS still propagates noisy reports under all-context sharing. This indicates that model scaling and report selection address different failure sources. Scaling improves local report quality; TRACE-MAS controls which reports propagate through the workflow.

The same observation explains why routing and verification are complementary. Retrieval-augmented generation and self-correction work already show that evidence selection and evidence checking are distinct sources of reliability [16, 17, 26]. Routing acts early by limiting which reports become visible. Verification then checks whether a visible claim is supported, contradictory, or risky. A verifier-only system can fail after unsupported reports have already shaped intermediate reasoning, while a routing-only system can be brittle when a false report is highly relevant. The practical design point is therefore not larger context or stronger verification alone, but a traceable report-sharing policy that decides what the aggregator sees.

### *6.3. Limitations*

TRACE-MAS currently relies on modular integrity metrics that we instantiate per task. Unsupported-claim labels, evidence precision, and contradiction scores are easiest to measure in controlled noisy-report settings, so these settings provide the cleanest trustworthiness analysis. The prototype also uses lightweight rankers for most experiments, with embedding and BM25 rankers serving as stronger alternatives. This choice keeps the method cheap enough for repeated MAS evaluation, but it leaves room for stronger report rerankers and claim-level verifiers. Finally, we evaluate self-revision with bounded revision budgets. Larger policy spaces can be explored in future work, but they should preserve the interpretability and trace records that make the method useful.

### *6.4. Implications for LLM-Era Information Access*

Our main implication is that trustworthy agentic systems need internal ranking layers. In traditional search, ranking governs which documents are shown. In agentic search and recommendation pipelines, ranking should also govern which agent-generated reports are shown to later agents. This view connects MAS design to foundation-model integrity: a system can be accurate on average yet untrustworthy if its internal evidence path is unsupported, unaudited, or overly sensitive to noisy reports. TRACE-MAS offers

one way to make this path inspectable by storing intermediate reports with source, visibility, risk, and cost records.

## 7. Conclusion

As LLM applications move from single-turn generation to agentic workflows, reliability increasingly depends on how intermediate evidence is shared inside the system. In LLM-based multi-agent systems (MAS), communication topology specifies possible information flow for MAS's dynamic workflow configuration, but it does not determine whether a downstream agent should trust, ignore, or verify each upstream report. In this paper, we conduct an in-depth analysis of this gap and argue that what downstream agents can read should be treated as an explicit design objective for trustworthy MAS. Thus, we introduced a trustworthiness-oriented framework (TRACE-MAS) for configuration-evolved MAS. TRACE-MAS treats internal report selection as an explicit design problem, routes reports, stores trace records, scores candidate configurations with TRACE Score, and revises routing, verifier placement, context budgets, and scoring weights through a bounded self-revision loop. Across standard benchmarks, noisy-report stress tests, cost-integrity analysis, transfer, and strong-backbone experiments, TRACE-MAS improves integrity metrics while maintaining competitive task performance. These results suggest that future MAS design should prioritize controllable report sharing, trace-based scoring, and auditable self-revision rather than treating agent-to-agent context sharing as a byproduct of communication topology.

## Data and Code Availability

The code implementation of TRACE-MAS and corresponding adopted datasets are publicly available in `https://github.com/albert-jin/TRACE-MAS`.

## Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Appendix A. Additional Experimental Notes

The main paper reports selected tables that best support the central argument. The accompanying artifact contains the complete result records used to construct the tables and figures. In particular, the released records include per-trial values for main results, preliminary exposure analysis, routing ablation, hallucination cascade stress tests, TRACE Score ablation, self-revision operator analysis, cost-integrity grids, clean-to-noisy robustness, transfer, qualitative case studies, judge sensitivity, low-dimensional trace analysis, Pareto geometry, cascade dynamics, and strong-backbone robustness.

## Appendix B. Benchmark, Backbone, and Reproducibility Details

This appendix expands the compact experimental setup in Section 5.1. We include these details because the paper compares standard task performance with trace-level integrity behavior, so the benchmark and backbone settings affect how the results should be read.

*Appendix B.1. Benchmark Details*

**Big-Bench Extra Hard (BBEH).** BBEH is a harder successor to Big-Bench Hard with 23 reasoning subtasks that cover logical and mathematical reasoning, language understanding, and spatial or structural reasoning [30]. We use the full BBEH setting for standard performance and BBEH-Mini, a stratified 460-example subset, for repeated noisy-report diagnostics.

**WorkBench.** WorkBench evaluates workplace agents in a sandbox with five databases, 26 read/write tools, and 690 tasks across analytics, calendar, CRM, email, project management, and multi-domain scenarios [31]. Success is outcome-based: the final sandbox state must match the target. We use it to test report visibility in tool-planning settings where plausible state summaries can mislead downstream agents.

**SWE-Bench-Lite and SWE-Bench-Verified.** SWE-Bench evaluates real GitHub issue resolution by asking agents to generate patches that pass repository tests [32]. We report SWE-Bench-Lite, a 300-instance subset, and SWE-Bench-Verified, a 500-instance human-validated subset [35]. These results test compatibility with software-engineering MAS evaluation, while the noisy-report experiments stress the paper's integrity claim more directly.

**Noisy-report variants.** For exposure-specific evaluation, we inject synthetic reports into selected BBEH-Mini and WorkBench traces rather than changing user queries or gold answers. The noise types are irrelevant, contradictory, fluent unsupported, partially correct, and mixed. This isolates whether routing and verification stop harmful internal reports before downstream reuse.

**Noisy-report construction protocol.** We execute the base MAS, record clean reports and decision points, insert noisy reports at worker, retriever, verifier, aggregator, or WorkBench tool-planning positions, and label support, contradiction, exposure, and reuse metadata. Unless stated otherwise, Noisy-BBEH-Mini uses a 30% injected-report ratio, with ratio stress at 10–50% fluent unsupported noise. Noisy-WorkBench mainly uses 30% partially correct or mixed misleading reports. The original task targets and evaluation metrics remain unchanged.

*Appendix B.2. Backbone Details*

**Claude-3.5-Sonnet (C-3.5S).** C-3.5S denotes Claude-3.5-Sonnet, used as a strong closed model in the fixed-backbone setting. In the tables, C-3.5S provides a compact label for rows where all agent roles use this backbone unless the method explicitly performs model selection.

**Qwen3-235B-A22B (Q-235B).** This is a mixture-of-experts Qwen3 model with 235B total parameters and 22B activated parameters [36]. We use it as a strong open-weight general backbone. The abbreviation emphasizes the active-parameter variant used in our experiment notes.

**Qwen3-Coder-480B-A35B (Q-480B).** Q-480B denotes Qwen3-Coder-480B-A35B, a coding-oriented mixture-of-experts backbone with 480B total parameters and 35B activated parameters [37]. We include it because software-engineering and tool-use MAS behavior may depend on coding-specialized capabilities rather than only on general reasoning strength.

**LLM-selection and strong-backbone settings.** The LLM-selection setting lets the system assign different models to different roles, such as solver, verifier, aggregator, or revision planner. The strong-backbone analysis uses GPT-5.5 and Claude Opus 4.6 labels from the experiment records to test whether simply scaling the underlying model removes report-sharing risk. The answer in our results is no: stronger models improve base capability, but unsupported reports and false consensus still require explicit routing and assessment.

*Appendix B.3. Backbone Access, API Settings, and Reproducibility*

For all backbone LLMs except the Claude-code-related strong-backbone runs, we accessed the models through their official API interfaces and kept the provider default runtime temperature unless a provider endpoint required an explicit value. The paper uses compact model aliases, such as C-3.5S, C-4S, C-4.5S, Q-235B, Q-480B, GPT-5.5, and Claude Opus 4.6; the released execution records store the corresponding provider-returned model strings, request timestamps, token usage, and run identifiers. Pricing follows the provider-published API price schedule at the time of access, while the paper reports token counts and latency rather than converting every run into a time-sensitive currency value.

For Claude-code-related backbone runs, access restrictions prevented a direct official-API call in our environment. We therefore used the OpenAI-compatible relay provided by `https://api.vectorengine.ai/`. The provider states that the routed models are genuine upstream models without model substitution or distillation. The request format follows the standard chat-completion interface; a representative call is:

```
curl https://api.vectorengine.cn/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer YOUR_API_KEY' \
  -d '{
  "model": "claude-sonnet-4.5",
  "messages": [{"role": "user", "content": "Say this is a test!"}],
  "temperature": 0.7
}'
```

The returned object contains the completion identifier, provider-returned model field, token-usage fields, generated message, and finish reason. We use the same logging fields for the backbone runs reported in the paper. Commercial API models are not fully weight-reproducible because providers may revise serving infrastructure, access policies, or prices. To reduce this source of variation, all compared systems follow the same task split, prompt contract, context budget, run count, token accounting, and evaluation protocol. Regardless of whether a run used an official API endpoint or the VectorEngine relay, we kept these controllable variables aligned with the EvoMAS baseline to control confounders and improve fairness.

*Appendix B.4. Hyperparameter Settings*

Unless an ablation explicitly changes the value, TRACE-MAS uses a report-exposure budget of $k = 2$ for top-$k$ routing, embedding-based relevance ranking for the default ranked policy, a cheap verifier for the main efficiency setting, and a self-revision budget of $T = 3$ rounds. The cost-integrity grid additionally evaluates $k \in \{1, 2, 3\}$ and all-context exposure, while the self-revision ablation evaluates $T \in \{0, 1, 3, 5\}$. Noisy-report experiments use a 30% injected-report ratio by default; the robustness sweep additionally uses 10%, 20%, 30%, and 50% injected fluent unsupported reports. The released TRACE-MAS configuration-search code keeps two parent configurations for selection, uses diversity weight 0.3 when diversity-aware selection is enabled, mutation probability 0.8, reward cost coefficient $\beta = 10^{-6}$, improvement threshold 0.05 for adding revised configurations to the pool, meta-model temperature 0.7, meta-model maximum output length 8192 tokens, role-level maximum output length 4096 tokens, timeout 600 seconds, and no retry by default. The implementation default integrity weights are 0.2 for evidence support, 0.1 for consistency, 0.2 for robustness, -0.3 for hallucination risk, -0.1 for unsupported claims, and -0.2 for contradiction score.

For reuse detection, the fixed cascade $\mathcal{D}_{\text{reuse}}$ first treats explicit report IDs or citations as reuse. Near-verbatim copying is then detected by normalized token-overlap with threshold $\tau_{\text{tok}} = 0.35$. Remaining exposed report-response pairs are matched by sentence-embedding cosine similarity with threshold $\tau_{\text{emb}} = 0.82$, and borderline pairs in $[\tau_{\text{low}}, \tau_{\text{emb}})$ with $\tau_{\text{low}} = 0.75$ are adjudicated by the verifier prompt. In the TRACE revision planner used by the released code, exposure-bounding revisions are triggered when noisy-report exposure exceeds 0.25 or the average number of visible reports exceeds 4; verifier insertion is triggered when hallucination risk exceeds 0.2, unsupported-claim signal is nonzero, or contradiction score exceeds 0.2. These thresholds are fixed across compared methods and are not tuned separately for individual baselines.

## Appendix C. Prompt Details

This appendix reports the core prompt templates used by the TRACE-MAS implementation. The templates are shown to make the role contract reproducible. Runtime fields such as task text, visible reports, routing records, and processor responses are filled by the prompt renderer.

*Appendix C.1. Worker Execution Prompt*

The worker prompt defines the basic agent interface. It asks each worker to solve the assigned task and to expose any required tool/function calls in a structured section.

```
You are a worker agent in a multi-agent system.

Your role is to analyze the given task and provide your analysis or solution.

**IMPORTANT**: If your solution involves tool/function calls, you MUST list them at the end in a section
     labeled "FUNCTION_CALLS:" using the format: domain.function.func(param="value")

Please work on the following task:

{{task}}
```

*Appendix C.2. Aggregation Prompt*

After exposure routing, the aggregator receives only the reports made visible by the selected routing policy. The prompt below specifies the final synthesis interface and preserves the structured tool-call convention.

```
You are an aggregator agent in a multi-agent system.

Your role is to synthesize and combine results from multiple worker agents to produce a final answer.

Task:
{{task}}

Results from worker agents:
{{worker_results}}

**IMPORTANT**: After analyzing the results, you MUST include a "FUNCTION_CALLS:" section listing ALL
    function calls needed to accomplish the task, using the format: domain.function.func(param="value")

Please analyze all the worker results and provide a final, consolidated answer.
```

*Appendix C.3. Verifier Prompt*

The verifier prompt is inserted by verifier-revision operators. It is the main prompt-level mechanism that converts routed reports and routing records into support, contradiction, risk, and report-weight guidance.

```
You are a TRACE-MAS integrity verifier.

Your task is to inspect upstream agent reports before final aggregation.

Original task:
{{task}}

Visible upstream reports:
{{reports}}

Routing metadata:
{{routing}}

Check the reports for:

1. Unsupported claims that are not justified by the visible evidence.
2. Contradictions between reports.
3. Overconfident claims under weak or noisy evidence.
4. Missing caveats or uncertainty.
5. Which reports should be trusted, down-weighted, or ignored by the final aggregator.

Return a concise verification report with these fields:

SUPPORTED_POINTS:
UNSUPPORTED_OR_RISKY_CLAIMS:
CONTRADICTIONS:
RECOMMENDED_REPORT_WEIGHTS:
FINAL_VERIFICATION_GUIDANCE:
```

*Appendix C.4. Candidate-Response Judge Prompt*

For SMoA-style selection, a judge prompt selects the most useful processor responses under a fixed output format. This prompt is not the full TRACE-MAS verifier, but it is part of the MAS prompt library and supports response ranking in predefined MAS baselines.

```
You are a judge agent in a Sparse Mixture-of-Agents (SMoA) system.

Your role is to evaluate and select the top-{{top_k}} best responses from multiple processors.

**Task**:
{{task}}

**Processor Responses**:
{{processor_responses}}

**Instructions**:
1. Carefully evaluate each response based on:
   - Correctness and accuracy
   - Completeness and thoroughness
   - Clarity and coherence
   - Unique insights and perspectives
2. Select the top-{{top_k}} responses that are most valuable for solving the task
3. Output ONLY the indices (1-indexed) of the selected responses, separated by commas

**Important**: Your output must be ONLY numbers separated by commas (e.g., "1,3,5"). Do not include any
    other text.

Selected response indices:
```

## Appendix D. Qualitative Case Study Summary

The qualitative cases show three recurring patterns. First, all-context routing often lets an unsupported but fluent report become visible to multiple downstream agents. Second, the aggregator may mistake repeated unsupported claims for consensus. Third, TRACE-MAS blocks the cascade either by down-ranking the risky report before exposure or by sending it to an integrity checkpoint before final aggregation. The full case-study notes include BBEH disambiguation, web-of-lies, multistep arithmetic, word sorting, and WorkBench email/project-management tasks.

| Case | Source task | Failure type | Gold answer | TRACE action |
|---|---|---|---|---|
| Q1 | BBEH Disambiguation QA id=64 | contradictory report | ambiguous | verify pronoun evidence before aggregation |
| Q2 | BBEH Disambiguation QA id=80 | contradictory report | squirrel | prefer semantically supported report |
| Q3 | BBEH Web of Lies id=18 | fluent unsupported report | yes, no, yes | down-rank incomplete truth-assignment chain |
| Q4 | BBEH Multistep Arithmetic id=116 | fluent unsupported report | 16624 | require operator-definition consistency check |
| Q5 | BBEH Word Sorting id=10 | false consensus | altered-alphabet ordering | route reports that explicitly use the altered alphabet |
| Q6 | BBEH Word Sorting id=108 | false consensus | custom-order sorting | preserve stepwise letter comparison evidence |
| Q7 | WorkBench Email id=295 | context flooding | sender, time, concrete request | rank subject, sender, datetime, and request sentence above background text |
| Q8 | WorkBench Project Management id=137 | context flooding | task id, task name, assignee | rank status and due date evidence above similar task rows |

Table D.14: Representative qualitative cases used for trace inspection. The cases are selected from BBEH and WorkBench task files and cover contradictory reports, fluent unsupported reports, false consensus, and context flooding.

## References


[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023.
URL https://arxiv.org/abs/2210.03629

[2] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: An open-ended embodied agent with large language models, arXiv preprint arXiv:2305.16291 (2023).
URL https://arxiv.org/abs/2305.16291

[3] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Dong, J. Tang, Agentbench: Evaluating llms as agents, arXiv preprint arXiv:2308.03688 (2023).
URL https://arxiv.org/abs/2308.03688

[4] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, B. Ghanem, Camel: Communicative agents for mind exploration of large scale language model society, arXiv preprint arXiv:2303.17760 (2023).
URL https://arxiv.org/abs/2303.17760

[5] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, J. Zhou, Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors, arXiv preprint arXiv:2308.10848 (2023).
URL https://arxiv.org/abs/2308.10848

[6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Li, E. Zhu, B. Jiang, L. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, Autogen: Enabling next-gen llm applications via multi-agent conversation, arXiv preprint arXiv:2308.08155 (2023).
URL https://arxiv.org/abs/2308.08155

[7] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, M. Sun, Chatdev: Communicative agents for software development, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024.
URL https://aclanthology.org/2024.acl-long.810/

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, Metagpt: Meta programming for a multi-agent collaborative framework, in: International Conference on Learning Representations, 2024.
URL https://arxiv.org/abs/2308.00352

[9] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: International Conference on Machine Learning, 2024.
URL https://arxiv.org/abs/2305.14325

[10] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Z. Liu, Y. Xu, M. Zhou, H. Xu, F. Huang, Towards reasoning in large language models via multi-agent peer review collaboration, arXiv preprint arXiv:2311.08152 (2023).
URL https://arxiv.org/abs/2311.08152

[11] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Jaward, B. Shang, Z. Gui, J. Fu, W. Wang, Autoagents: A framework for automatic agent generation, arXiv preprint arXiv:2309.17288 (2023).
URL https://arxiv.org/abs/2309.17288

[12] J. Ye, et al., Mas-gpt: Training llms to build llm-based multi-agent systems, arXiv preprint arXiv:2503.03686 (2025).
URL https://arxiv.org/abs/2503.03686

[13] S. Hu, C. Lu, J. Clune, Automated design of agentic systems, arXiv preprint arXiv:2408.08435 (2024).
URL https://arxiv.org/abs/2408.08435

[14] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, C. Wu, Aflow: Automating agentic workflow generation, in: International Conference on Learning Representations, 2025.
URL https://arxiv.org/abs/2410.10762

[15] Y. Hu, M. Trager, Y. Zhang, Y. Zhang, S. Yang, W. Xia, S. Soatto, Evomas: Evolutionary generation of multi-agent systems, in: International Conference on Machine Learning, 2026. arXiv:2602.06511.

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: Advances in Neural Information Processing Systems, 2020.
URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[17] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-rag: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations, 2024.
URL https://arxiv.org/abs/2310.11511

[18] P. Manakul, A. Liusie, M. J. F. Gales, Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.
URL https://aclanthology.org/2023.emnlp-main.557/

[19] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, Factscore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.
URL https://aclanthology.org/2023.emnlp-main.741/

[20] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, J.-R. Wen, Halueval: A large-scale hallucination evaluation benchmark for large language models, arXiv preprint arXiv:2305.11747 (2023).
URL https://arxiv.org/abs/2305.11747

[21] S. Robertson, H. Zaragoza, The probabilistic relevance framework: Bm25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389. doi:10.1561/1500000019.

[22] O. Khattab, M. Zaharia, Colbert: Efficient and effective passage search via contextualized late interaction over bert, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020, pp. 39–48. doi:10.1145/3397271.3401075.

[23] S. Yuan, K. Chen, J. Zhang, B. Yang, Z. Peng, J. Chen, Z. Wang, X. Liang, Y. Chen, Evoagent: Towards automatic multi-agent generation via evolutionary algorithms, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics, 2025.
URL https://aclanthology.org/2025.naacl-long.315/

[24] C. J. C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, G. Hullender, Learning to rank using gradient descent, in: Proceedings of the 22nd International Conference on Machine Learning, 2005, pp. 89–96. doi:10.1145/1102351.1102363.

[25] C. J. C. Burges, R. Ragno, Q. V. Le, Learning to rank with nonsmooth cost functions, in: Advances in Neural Information Processing Systems, 2006, pp. 395–402.
URL https://proceedings.neurips.cc/paper/2006/hash/af44c4c56f385c43f2529f9b1b018f6a-Abstract.html

[26] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv preprint arXiv:2401.15884 (2024).
URL https://arxiv.org/abs/2401.15884

[27] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, Ragas: Automated evaluation of retrieval augmented generation, arXiv preprint arXiv:2309.15217 (2023).
URL https://arxiv.org/abs/2309.15217

[28] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, G-eval: Nlg evaluation using gpt-4 with better human alignment, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.
URL https://aclanthology.org/2023.emnlp-main.153/

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging llm-as-a-judge with mt-bench and chatbot arena, in: Advances in Neural Information Processing Systems, 2023.
URL https://arxiv.org/abs/2306.05685

[30] M. Kazemi, et al., Big-bench extra hard, arXiv preprint arXiv:2502.19187 (2025).
URL https://arxiv.org/abs/2502.19187

[31] S. Styles, et al., Workbench: A benchmark dataset for agents in a realistic workplace setting, arXiv preprint arXiv:2405.00823 (2024).
URL https://arxiv.org/abs/2405.00823

[32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, Swe-bench: Can language models resolve real-world github issues?, in: International Conference on Learning Representations, 2024.
URL https://arxiv.org/abs/2310.06770

[33] Y. Du, P. Zhang, K. Li, S. Tan, X. Wang, Z. Lin, Y. Fang, Z. Tang, S. Yao, W. Li, Z. Chen, W. Chen, Multi-agent collaboration via cross-team orchestration, in: Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 10386–10406.
URL https://aclanthology.org/2025.findings-acl.541/

[34] Y. Li, X. Li, Y. Chen, H. Tang, Y. Wang, J. Liu, D. Gao, Smoa: Improving multi-agent large language models with sparse mixture-of-agents, in: Advances in Knowledge Discovery and Data Mining, Springer Nature Singapore, 2025, pp. 54–65. doi:10.1007/978-981-96-8180-8_5.
URL https://doi.org/10.1007/978-981-96-8180-8_5

[35] OpenAI, Introducing swe-bench verified, https://openai.com/index/introducing-swe-bench-verified/ (2024).

[36] Qwen Team, Qwen3 technical report, arXiv preprint arXiv:2505.09388 (2025).
URL https://arxiv.org/abs/2505.09388

[37] Qwen Team, Qwen3-coder-480b-a35b-instruct, https://huggingface.co/Qwen/Qwen3-Coder-480B-A35B-Instruct (2025).